

COSMOLOGICAL DATA ANALYSIS USING ARTIFICIAL INTELLIGENCE TECHNIQUES



Bachelor's Final Thesis Physics

Alberto Nieto Cardoso

Supervised by Pietro Vischia

Co-supervised by Georgios Alestas



University of Oviedo



Contents

1	Introduction	1
2	Theoretical Framework	3
2.1	Basic Machine Learning Concepts	3
2.2	Redshift in Cosmology	17
3	The Dataset	20
3.1	Data Collection and Preparation	20
3.2	Data Preprocessing	25
4	Problem Modelling	31
4.1	Global Structure	31
4.2	Model Architectures	34
5	Results	38
5.1	Comparative Analysis	38
5.2	Challenges and Issues	46
5.3	Discussion	49
6	Conclusions	51
7	References	53
8	Glossary	56

Chapter 1

Introduction

Artificial intelligence models have ceased to be a novelty for several years now. So much so that the world of machine learning has been advancing by leaps and bounds since the immense potential of this field was discovered, with a vast number of approaches having already been explored to train these models for tasks where no one imagined they would find a place.

This work aims to contribute to this trend, starting from a problem of great importance in theoretical physics where this approach could be of great interest: accurately estimating the cosmological redshift z . This scalar parameter takes values in a continuous space and will be inferred from astronomical spectra using several models.

These spectra are neither images nor scalar features taking a single value (such as a person's height or weight); instead, for each sample, two huge data channels contain the flux or amount of light measured across wavelength bins for a given astronomical object.

Our goal is therefore to construct a neural network capable of identifying the physical patterns inherent to redshift within the spectra, ignoring the components unsuited for redshift regression that act as noise. Such noise dominates most astronomical spectra, as only specific wavelength ranges yield reliable information for estimating the parameter.

Throughout this report, the necessary theory and procedures to arrive at the final model will be presented. Starting from scratch, we begin with simpler models and progressively develop more refined architectures tailored to our specific task, using generalization capability on large datasets as the primary benchmark.

Finally, we discuss the specific challenges encountered during development and the advantages these methods offer despite those limitations, evaluating whether they represent a valid and useful alternative to traditional approaches currently in use.

Note that this work is strongly supported by code (**Jupyter Notebooks**), which is necessary for a complete understanding of the project implementation. The code repository is available on [GitHub](#) and can be viewed using any standard **Python** development environment, such as **Visual Studio Code**.

Chapter 2

Theoretical Framework

2.1 Basic Machine Learning Concepts

[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

—**Arthur Samuel**, 1959

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

—**Tom Mitchell**, 1997

The vast majority of definitions, typologies, and implementation mechanisms in **Python** presented below have been drawn from the book [1, *Deep Learning*], which synthesizes basic and intermediate theory of this field in a very accessible manner.

That being said, **machine learning** is a discipline dating back to the 1950s, when computational methods capable of emulating human learning processes were first explored. During that era, the basic concept of the perceptron was defined and the theoretical foundations of neural networks were established, leveraging the rise of digital computers to implement linear classification models.

In the last decade, machine learning has undergone a true revolution thanks to the development of specialized architectures, such as famous *LLMs* (Large Language Models), trained on vast text volumes to understand, predict, and generate coherent language (ChatGPT), and *convolutional networks*, which are crucial for image processing and generation (Midjourney). Both implementations serve as inspiration for the development of this work.

2.1.1 Fundamentals

First, we need to define what a **model** is and what modeling a problem consists of. A computational model is defined as a mathematical representation establishing relationships between input and output variables, enabling behavior predictions from data. In the context of machine learning, a model learns statistical patterns from a training set to generalize to unseen examples.

This work focuses particularly on **supervised** machine learning. That is, given a **training dataset** as the collection $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^m$ represents the m features of **sample** i and $y^{(i)}$ the target value, the **loss function** $\mathcal{L}(\theta) = \ell(f_\theta(\mathbf{x}), y)$ is defined as the distance in a certain *metric* ℓ between the value predicted by the model f (with parameters θ) and the target value.

The learning process consists of tuning the network's internal parameters by minimizing this loss function, which quantifies the discrepancy between predictions and true labels. Each iteration of this process, corresponding to the network completing one full pass through the training dataset, is called an **epoch**.

In addition to the training set, an independent **validation** dataset can be used during each epoch, which serves to validate the network's performance on unseen samples. Evaluating the network on this set allows us to monitor the model's regression capacity.

Under this paradigm, an **artificial neural network** (ANN) is conceived as an assembly of processing units, called artificial neurons, organized in **layers** and connected by **weights**. Each neuron receives an *input vector*, computes a linear combination of its elements, and applies a non-linear activation function to produce an *output signal*. This assembly is structured in an architecture where layers succeed each other without feedback, constituting what is known as a *feedforward network* that propagates information in one direction only.

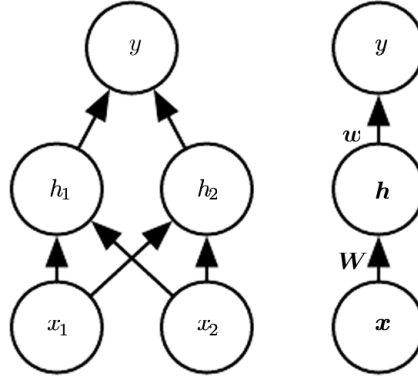


Figure 1: Diagram of a feedforward network showing input $\mathbf{x}$, hidden layer $\mathbf{h}$, and output $\mathbf{y}$, connected via weights.

To speak precisely, one must distinguish between model **parameters**, which correspond to the weights and biases adjusted automatically during optimization, and **hyperparameters**, which must be set prior to training and control the representation capacity and convergence dynamics of the algorithm, such as the number of layers and neurons in the network.

We now define the fundamental unit of such models. The **perceptron** is the simplest model of an artificial neuron. It is defined as a linear function followed by a step activation function, taking an input vector $\mathbf{x} = [x_1, x_2, \dots, x_n]$ and producing a binary output $y \in \{-1, 1\}$.

Mathematically, a weight vector $\mathbf{w} = [w_1, w_2, \dots, w_n]$ and a bias b are introduced, forming the linear combination

$$z = \mathbf{w}^\top \mathbf{x} + b,$$

and subsequently applying the step function

$$y = \begin{cases} 1, & \text{if } z \geq 0, \\ -1, & \text{if } z < 0. \end{cases}$$

With this definition, a single perceptron separates the input space via a hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$, classifying points on one side as positive and on the other as negative.

The training process of this simple model is based on the perceptron learning rule, which iteratively adjusts weights and bias to correct misclassifications in a linearly separable dataset. In each iteration and for each sample $(\mathbf{x}^{(i)}, y^{(i)})$, the parameters are updated according to

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y^{(i)} - \hat{y}^{(i)}) \mathbf{x}^{(i)}, \quad b \leftarrow b + \eta (y^{(i)} - \hat{y}^{(i)}),$$

where $\hat{y}^{(i)}$ is the current prediction, $y^{(i)}$ the true label, and η the **learning rate**.

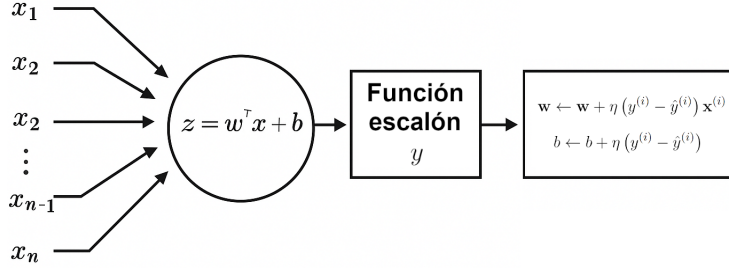


Figure 2: Visual diagram of the perceptron concept and parameter updating.

2.1.2 The Multilayer Perceptron

The **multilayer perceptron (MLP)** is conceived as a set of processing units transforming information through chained linear and non-linear operations. In early developments, each neuron performed a weighted sum of inputs followed by an activation function, enabling ANNs to capture non-linear patterns.

This concept proved essential when single-layer perceptrons were shown to be incapable of solving non-linearly separable classification problems, such as the famous **XOR** problem [32].

Thus, the notion of **hidden layers** located between input and output layers was introduced, allowing the network to approximate arbitrary functions provided it had sufficient units and depth. This word, "sufficient", is the main challenge when putting this method into practice, as no theoretical result can determine beforehand how many neurons are needed to solve a specific problem.

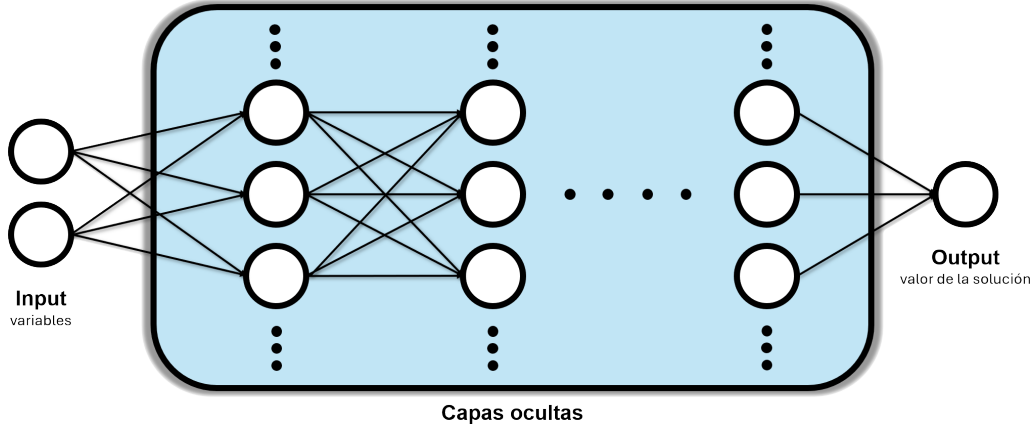
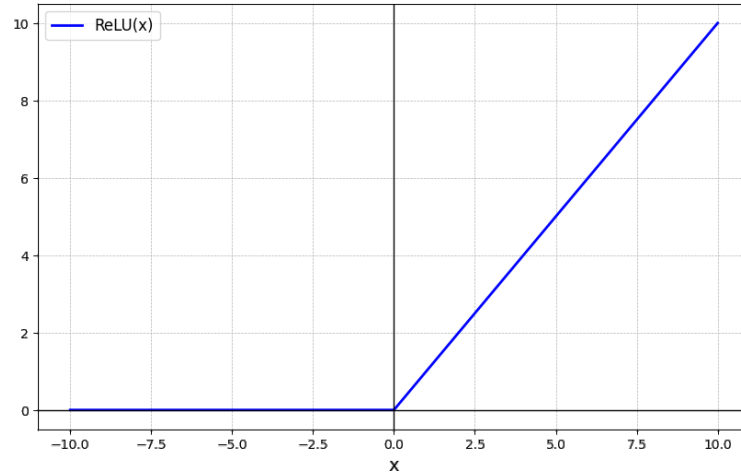


Figure 3: Generic diagram of MLPs with fully connected layers.

Within feedforward networks, the MLP is identified as the elemental subclass. The input layer receives original feature values, hidden layers generate intermediate representations of higher abstraction, and the output layer emits the final prediction. Each linear layer is expressed as $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, with $\mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)})$ the **activation function** and ℓ the layer index, where ϕ can be, for instance, the famous *Rectified Linear Unit* (ReLU) function.

Figure 4: $\text{ReLU}(x)$, activation function used in this work [15].

$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ contains in each row the weights connecting all neurons of layer $\ell - 1$ to a neuron of layer ℓ , while $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$ is a vector whose components are the biases of each neuron in layer ℓ .

From the function approximation perspective, it was proven that networks structured in this way, with a hidden layer containing a sufficiently large number of neurons and employing non-linear activations, possess the property of *universal approximation*. Established in the 1980s, this property underpins the ability of these networks to model complex relationships between input and output variables.

Theorem (Universal Approximation): Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a continuous function defined on a compact subset $K \subset \mathbb{R}^n$. Then, for every $\varepsilon > 0$, there exists a single-hidden-layer neural network (i.e., a multilayer perceptron with arbitrary non-linear activation) with a finite number of hidden neurons such that, denoting its output by $\hat{f}(x)$,

$$\sup_{x \in K} |\hat{f}(x) - f(x)| < \varepsilon.$$

To minimize the aforementioned loss function, the error **backpropagation** algorithm is used, updating all model parameters using a gradient descent-based **optimizer**.

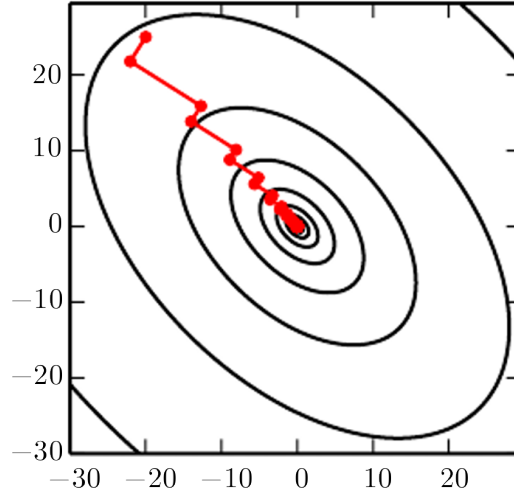


Figure 5: Visual diagram of error progression towards lower loss function contours in high-dimensional space.

In this way, training a neural network (whether MLP or the architectures discussed next) consists of three fundamental phases:

1. Forward propagation: Inputs $\mathbf{x}$ are passed into the input layer, and the output of each linear layer is computed successively:

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)}),$$

until obtaining the final prediction $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$. Next, the loss function $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ is evaluated.

2. Backpropagation: The gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ is computed. Applying the chain rule yields, for each weight and bias:

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}} \frac{\partial \mathbf{z}^{(\ell)}}{\partial W^{(\ell)}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}}.$$

3. Parameter update: Once gradients are available, the optimizer applies the gradient descent rule:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t),$$

where $\boldsymbol{\theta}$ aggregates all weights $W^{(\ell)}$ and biases $\mathbf{b}^{(\ell)}$, and η is the **learning rate**.

It has been observed that a learning rate that is too high induces oscillations and instability, whereas one that is too low slows down training. To dynamically adjust this parameter, **schedulers** are incorporated to gradually reduce the learning rate in various ways. In our model implementations, a gradual reduction of the learning rate based on validation error plateaus was selected.

2.1.3 Convolutional Neural Networks

Convolutional neural networks (CNNs) process data structured on a two-dimensional grid, such as images, where each grid point corresponds to a pixel with an intensity value or channel vector.

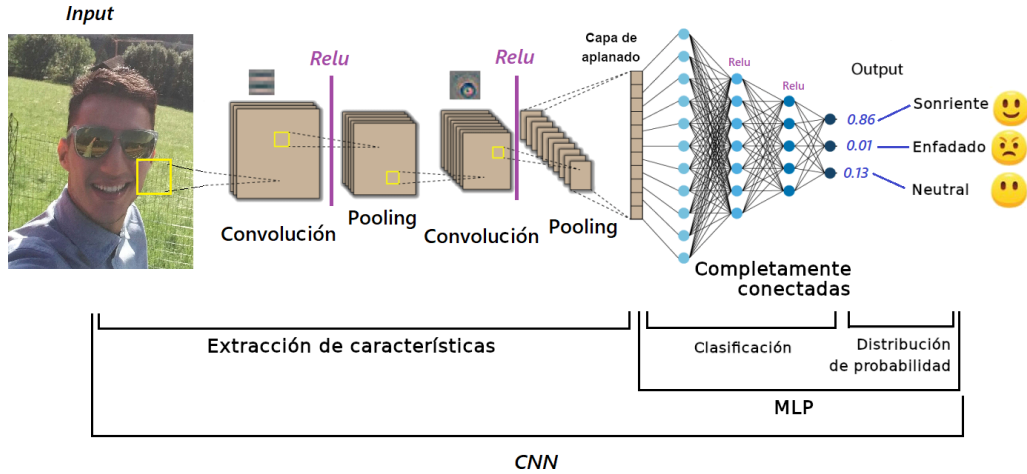


Figure 6: CNN structure scheme. The input image is represented as a matrix of values (intensities or RGB vectors) organized in rows and columns, to which a series of filters is applied to extract features abstracted by an MLP in a lower-dimensional space [7].

These networks rely on the concept of *local connectivity*, which dictates that each convolutional neuron observes only a small window of the grid, called the **receptive field**, greatly reducing the number of parameters compared to a fully connected MLP layer.

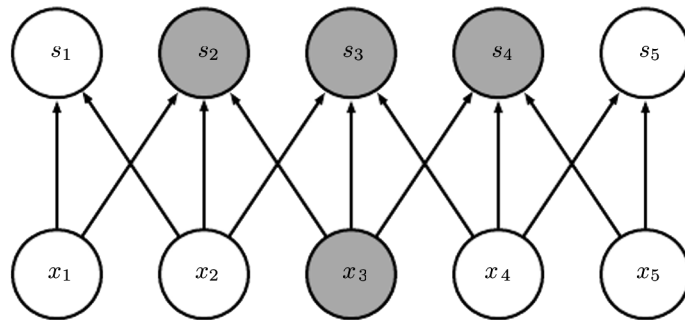


Figure 7: Each unit in the convolutional layer is connected only to a local block of the previous layer.

We define a **kernel** or filter as a small tensor of learnable parameters used to extract local features from an input. Intuitively, the kernel acts as a local pattern detector (edges, textures, shapes), learning during training to respond to the most relevant features of the data.

More formally, a kernel of dimensions $C_{\text{in}} \times k \times k$ is a set of weights

$$K = \{K_{c,u,v} \in \mathbb{R} \mid c = 1, \dots, C_{\text{in}}, \quad u, v = 0, \dots, k-1\}$$

and a *scalar bias* $b \in \mathbb{R}$, applied to each input position X to produce output Y via discrete convolution:

$$Y_{i,j}^{(f)} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{c,u,v}^{(f)} X_{i+u,j+v}^{(c)} + b^{(f)}, \quad f = 1, \dots, C_{\text{out}}.$$

Thus, each output channel f is the global response of filter $K^{(f)}$ across the input. Here k is the kernel **size** (determining local receptive field), C_{in} is the number of **input channels**, and C_{out} is the number of **output channels**.

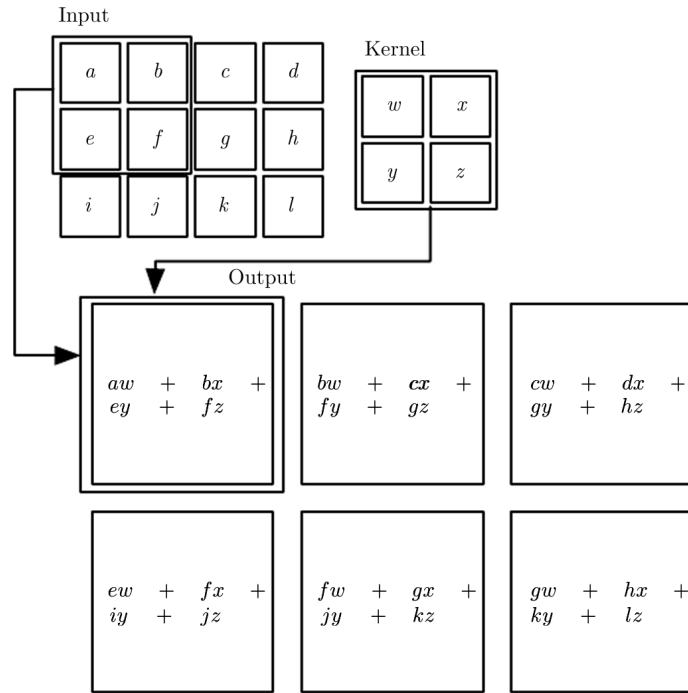


Figure 8: The same filter K is applied or *slid* across the image, generating an activation map that indicates the presence of the associated feature.

This leads to the concept of *weight sharing*, where a single set of parameters K and b

is reused across all positions. This offers two fundamental advantages: *spatial invariance* (the detector responds to the same feature regardless of where it appears) and *parameter reduction* (instead of learning unique weights for each position, we learn only k^2 per filter, enhancing efficiency and improving generalization on unseen data).

Additional parameters are associated with discrete convolution. One of them is **stride**, controlling filter displacement.

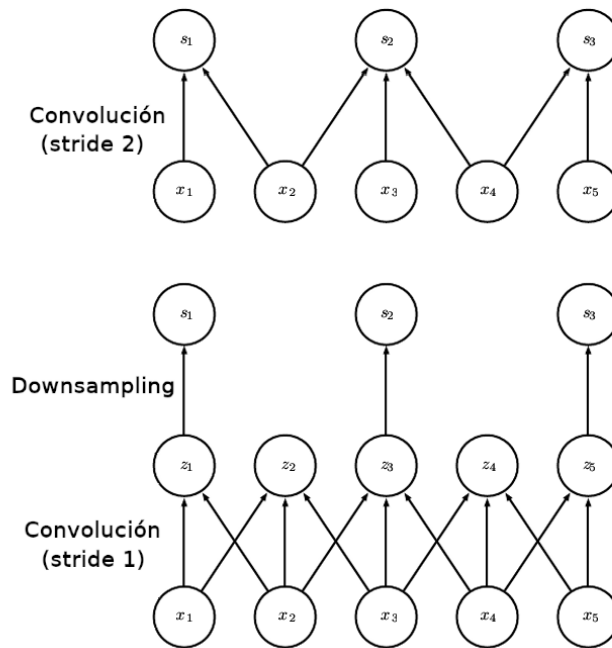


Figure 9: Applying a convolution with stride equal to 2 (top, one output per two inputs) is equivalent to applying stride 1 followed by downsampling (bottom).

There is also **padding**, defined as the number of elements or pixels added around output features. Specifically, zero padding adds zero elements. Figure 10 illustrates how padding controls network depth.

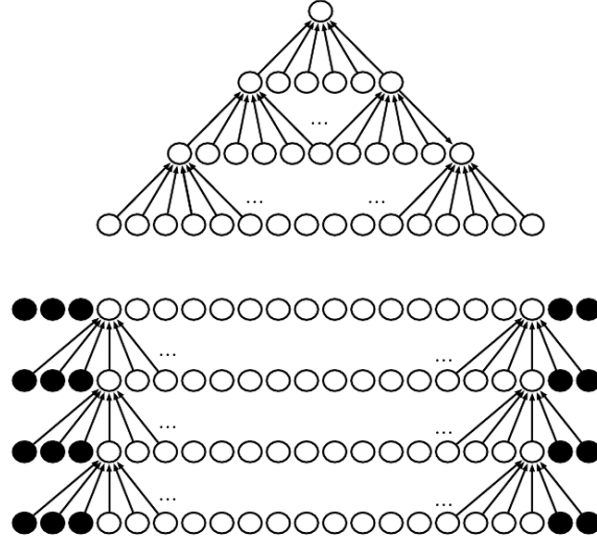


Figure 10: Padding detail. By adding five implicit zeros in each layer, we prevent feature representation shrinking with network depth, allowing arbitrarily deep convolutional networks.

Furthermore, **pooling** is a spatial downsampling operation applied over windows of size $p \times p$ in each activation map. In *max-pooling*, which we use and is most common, output Y at position (i, j) is defined as:

$$Y(i, j) = \max_{\substack{1 \leq u \leq p \\ 1 \leq v \leq p}} X(s i + u - 1, s j + v - 1),$$

where s is pooling *stride* and X is the input. This selects the highest value in each window, retaining strong features while discarding lower-intensity information.

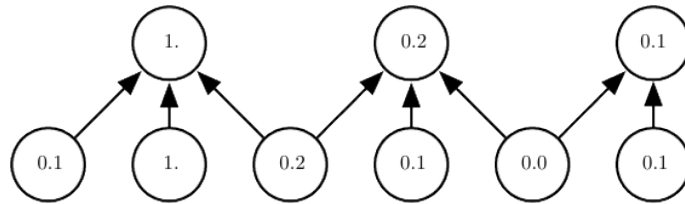


Figure 11: Visual example of max-pooling: taking maximum value within each block to build a smaller output map.

2.1.4 The Transformer Architecture

Transformers (TRA) are attention-based models that have revolutionized natural language processing. Rather than processing words sequentially, they represent each token

(minimal text unit) as a point in a continuous space via embeddings. The general explanation provided here and its implementation for our task are closely tied to the paper [3, *"Attention Is All You Need"*].

An **embedding**, in natural language processing, is simply a way to convert words or text fragments into numerical vectors so the model can process them mathematically. Thus, instead of treating each word as a discrete identifier (an integer), we assign each **token** a point in a continuous high-dimensional space, so that words with similar meanings (such as "France" and "Spain") lie close together, while unrelated words are far apart.

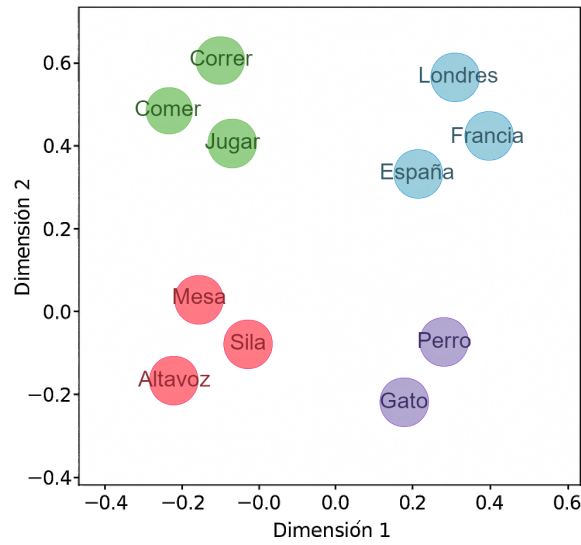


Figure 12: Visualization of the embedding concept in the context of natural language.

Transformers determine which text parts are relevant using a mechanism similar to internal Q&A, called **attention**.

Starting from a token sequence, the model creates three properties for each word, named Q (query), K (key), and V (value) in literature.

Each query is compared to all keys to measure affinity. Higher affinity leads to greater attention assigned to the corresponding value. Thus, processing "eats", the model pays high attention to "cat" and "fish", and less to "the" or "a".

This attention mechanism allows the model to connect distant spectral features, such as recognizing intensity peaks and learning relative distances between them.

To further enrich this process, the Transformer runs multiple attention mechanisms

in parallel, called **attention heads**. Each head acts as an interlocutor posing queries and keys to find different relationships. The model then aggregates all information into a unified representation.

Figure 13 summarizes this concept, where h is the number of attention heads.

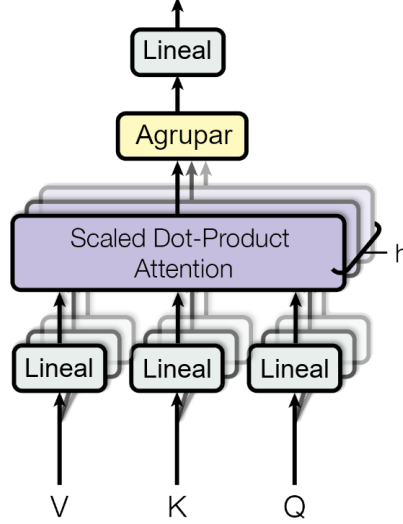


Figure 13: Multi-head attention mechanism operating in parallel.

Since the architecture does not process sequences sequentially, sinusoidal **positional encodings** (PE) are added to embeddings to indicate token positions (beginning, middle, end):

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right),$$

where pos is token position.

Once attention is applied, each position is refined independently with a two-layer feed-forward network with ReLU activation.

In language processing, these mechanisms enable translation, question answering, and text generation. At a conceptual level, training is identical to previously discussed models. A classic example is a chatbot like ChatGPT or DeepSeek.

Likewise, we can apply this to astronomical spectra, which can be interpreted as sequence data mapped into high-dimensional space.

Initial experiments began with MLP and CNN models, which focus mostly on local feature extraction. As shown below, consistently calculating cosmological redshift requires

understanding not only local spectral regions but also global relative relationships across the spectrum.

2.1.5 Regularization

When training a model, we aim for it to learn general patterns without memorizing specific details or being overly superficial. The term **overfitting** describes when a model learns the training set so thoroughly that it overfits its parameters to training noise, drastically reducing generalization to new data.

Conversely, **underfitting** occurs when the model is too simple or data too sparse to capture main trends, resulting in high training and test errors regardless of training time.

Imagine training a CNN with a million images of the exact same coffee mug to classify whether an image contains a coffee mug. Given the training mug, it will classify it correctly, but given a mug of a different color, it may fail because it did not abstract the general concept of a mug from the provided data.

Techniques against underfitting involve building a better dataset and expanding model capacity. Overfitting study goes further: with a large, complex model capable of learning minor patterns, achieving better performance on unseen data becomes challenging. Convolutional layers help in this regard.

A key technique used here is **dropout** [12], which randomly deactivates a fraction p of neurons in a layer by setting their output to 0, typically between linear layers during forward pass in training.

We also use **weight decay**, acting as a regularizer penalizing weight magnitudes to improve generalization [28].

2.2 Redshift in Cosmology

2.2.1 Definition and Conceptualization

Before defining redshift in cosmology, we establish notation.

Cosmic time (t) is the common time coordinate for observers at rest. t_E is emission time and t_R reception time.

Scale factor $R(t)$ measures relative spatial expansion ($R(t_0) = 1$ at present).

Recession velocity v_R is the speed at which a galaxy recedes due to cosmic expansion.

Comoving distance L is constant separation in expanding coordinates.

The **redshift** z is defined as the relative wavelength shift of a spectral line between observed wavelength λ_{obs} and rest-frame wavelength λ_0 :

$$z = \frac{\lambda_{\text{obs}} - \lambda_0}{\lambda_0}, \quad (2.1)$$

and relates to scale factor via:

$$1 + z = \frac{R(t_R)}{R(t_E)}, \quad (2.2)$$

resulting from spatial expansion along light's path.

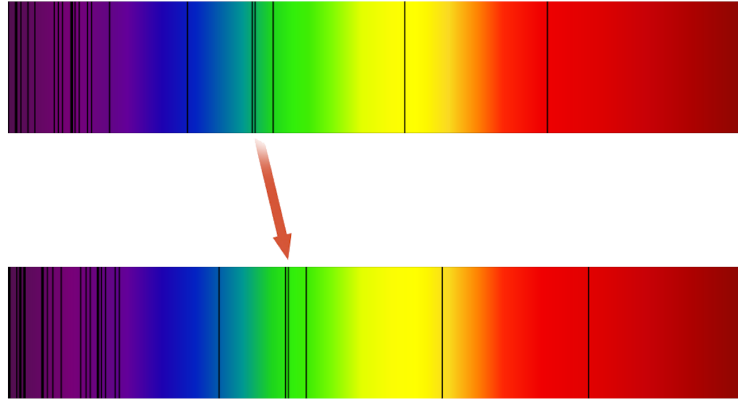


Figure 14: The farther away a source is, the greater redshift its spectrum exhibits due to cosmic expansion.

For small $z \ll 1$, redshift acts as a non-relativistic Doppler effect:

$$z \approx \frac{v}{c}, \quad (2.3)$$

valid for $v \ll c$.

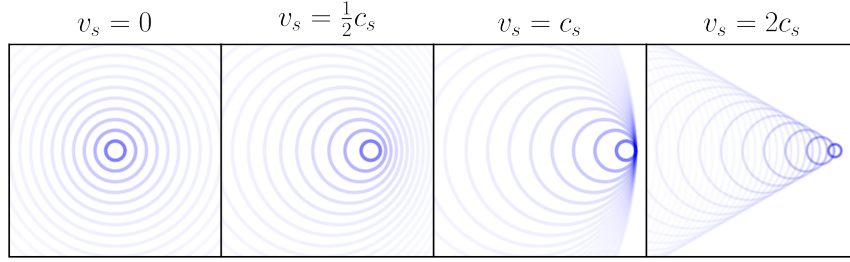


Figure 15: Taking c_s as speed of sound, Doppler effect magnifies when a sound source moves rightward with increasing speed relative to c_s [5]. Though acoustic here, the same occurs for a luminous object whose recession speed approaches light speed.

Hubble's Law states that recession velocity is proportional to distance L :

$$v = H(t)L,$$

where $H(t)$ is the expansion rate. At present time t_R , $H(t_R)$ is the Hubble constant. Combining with Doppler approximation yields linear redshift-distance relation for $z \ll 1$:

$$z \approx \frac{H(t_R)L}{c}. \quad (2.4)$$

Note that measured redshift combines expansion with gravitational redshift [8] or gravitational lensing [9], adding noise when inferring parameters.

2.2.2 Importance of Redshift

Redshift is one of modern cosmology's most powerful tools, directly indicating electromagnetic wavelength stretching from emission to reception.

From these relations, we extract recession velocities of galaxies and quasars (2.3), estimate comoving and luminosity distances, and infer parameters governing global cosmic dynamics and geometry.

Using General Relativity, redshift reveals dark matter and dark energy concepts, and calibrates the Hubble constant (achieving near 1% precision via Type Ia supernovae), highlighting the "Hubble tension" when compared with Cosmic Microwave Background values.

Redshift connects general relativity with astronomical observation.

2.2.3 Measurement Techniques

Beginning with Vesto Slipher studying Andromeda's spectrum at Lowell Observatory using a high-dispersion prism and short-focus camera, early spectrograms revealed blueshifted spectral lines (approaching at 300 km/s) and subsequent redshifts in 14 spiral nebulae [4].

This launched astronomical **spectroscopy**. Today's automated process collects light, passes it through a spectrograph (diffraction gratings), exposes detectors, and calibrates against emission lamps (H, Hg, Ne, Ar) to identify absorption and emission lines, computing z via spectral line shifts.

An astronomical object emits a continuous spectrum. Passing through gas, specific wavelengths are absorbed (absorption spectrum). Absorbed energy is re-emitted in all directions at characteristic wavelengths (emission spectrum).

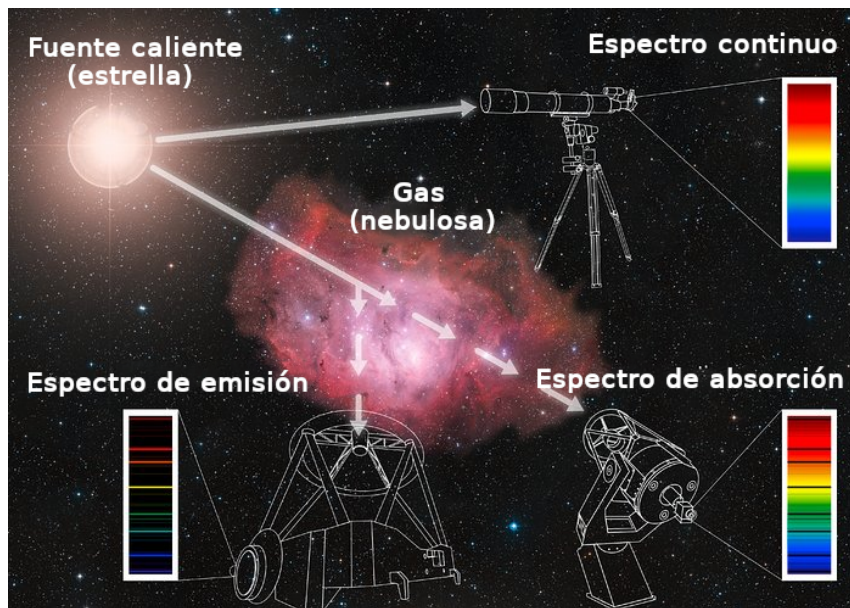


Figure 16: Spectroscopy is key to measuring these spectra accurately; working at nanometer scales requires small pixel sizes to resolve spectral lines.

For formal definitions and equations in this section, we refer to the book [2, Chapter 6].

Chapter 3

The Dataset

3.1 Data Collection and Preparation

3.1.1 Main Source

The core bulk of this project relies on data provided by the **Sloan Digital Sky Survey**, abbreviated as **SDSS** [16]. This immense astronomical mapping survey combines multi-band imaging and fiber-optic spectroscopy to generate catalogs of over 1 billion objects and 4 million spectra, with a scientific reach spanning from large-scale cosmic structure studies to stellar physics in our Milky Way.

Its primary purpose is to provide a uniform, high-volume dataset enabling multi-scale studies of matter distribution in the Universe. The photometric survey was designed to detect galaxies, quasars, and stars down to established magnitude limits, facilitating statistical studies of angular and photometric correlation over large cosmic volumes.

Through spectroscopic mapping, SDSS has measured redshifts for millions of galaxies and quasars, mapping the 3D structure of the Universe and identifying the characteristic scale of *baryon acoustic oscillations* (BAO)—key to constraining the *cosmic expansion history*—all made open and transparently accessible worldwide.

The primary telescope provides excellent image quality over a broad field of view with minimal geometric distortions across the 3000–10600 Å wavelength range, which forms the spectral domain of our work [17].

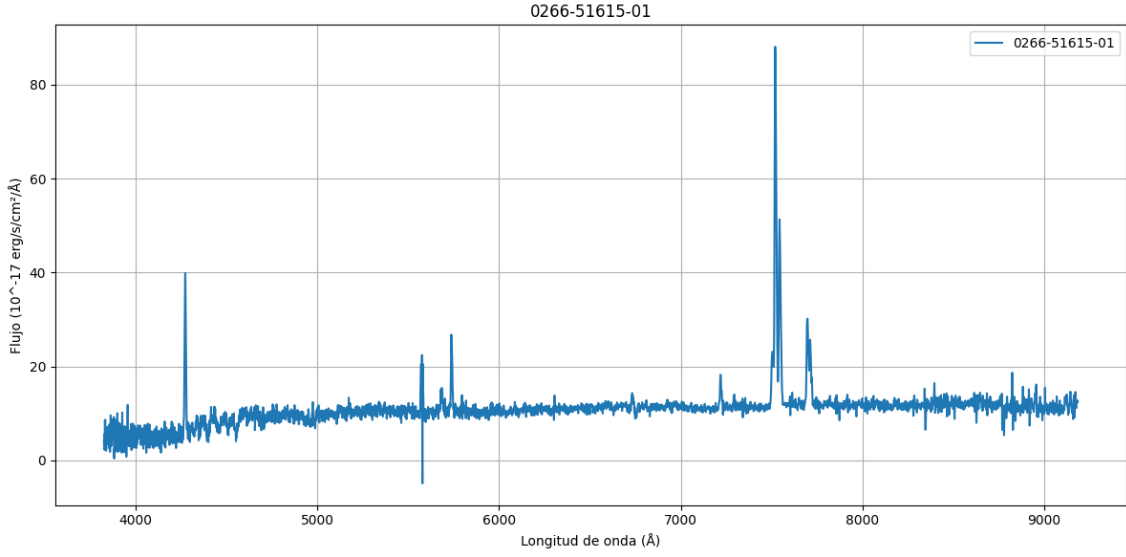


Figure 17: Spectrum of galaxy 0266-51615-01 from SDSS.

In spectroscopic mode, the telescope focuses light onto custom-drilled aluminum plates, where each perforation aligns with a target object and connects to an optical fiber. From there, a *spectroscopic pipeline* (a structured suite of automated algorithms and procedures) processes raw measurements into calibrated flux and wavelength data.

The camera incorporates five filters designed to maximize spectral coverage and minimize atmospheric contamination:

- **u (355 nm)**: avoids the Balmer jump.
- **g (468 nm)** and **r (616 nm)**: broad bands avoiding atmospheric *OH* sky lines at 5577 Å.
- **i (748 nm)**: cuts off before telluric *OH* absorption bands.
- **z (893 nm)**: avoids water absorption lines.

In the database, these filter magnitudes form a vector $\mathbf{x} = (m_u, m_g, m_r, m_i, m_z)$ useful for photometric redshift estimation using high-order optimized polynomials as demonstrated by [10].

- an uncertainty metric **zErr** derived from cross-correlation peak width and line-fitting covariance matrices
 - a flag parameter **zWarning** [23] alerting to potential issues (low signal-to-noise, overlapping lines, sparse reliable lines)
- ensuring high dataset reliability.

3.1.2 File Structure

Data retrieval utilizes two dedicated archive servers: the *Catalog Archive Server* (CAS) [20], powered by **SQL** Server [31], which supplies object metadata via SkyServer and CasJobs, as well as pointers to raw data on the *Science Archive Server* (SAS) [21]. **SQL** technology allows intuitive database queries to select and structure target objects.

Spectroscopic reduction is performed by the *spec2d* pipeline [24], transforming 2D raw exposures into 1D flux- and wavelength-calibrated spectra with noise estimates and filter transfer functions. Multiple exposures of the same field are stacked to improve signal-to-noise ratio, outputting final **.fits** files containing spectra, variance, and calibration metadata per fiber.

The **FITS** (Flexible Image Transport System) format [22] organizes data into header-data units (HDUs). The header stores essential metadata (extension type, object identifiers, observation timestamps), while the data block contains pixel arrays or binary tables.

In SDSS, primary HDUs contain overall observation metadata, followed by structured HDUs holding 1D spectra, variances, and quality metrics.

Specifically, a typical SDSS **.fits** file contains:

- HDU[0]: general observation metadata.
- HDU[1]: core vectors, including **flux** and **loglam** ($\log_{10}$ wavelength).
- HDU[2]: object table with unique **objid**, measured **redshift**, and classification metadata.
- HDU[3]: detected spectral line parameters linked to the target object.

3.1.3 Other Sources

The **NASA/IPAC Extragalactic Database (NED)** [25] is a centralized repository operated by Caltech’s IPAC and funded by NASA. It aggregates extragalactic object data (galaxies, quasars, radio, X-ray, and IR sources) from multi-wavelength catalogs and literature. NED serves as a master cross-identification catalog, providing positions, redshifts, and bibliographic references.

Querying NED’s relational SQL interface enabled collecting approximately 90,000 astronomical spectra [26].

3.2 Data Preprocessing

This section describes the end-to-end data pipeline from raw downloads to finalized tensor formats used for neural network training and evaluation.

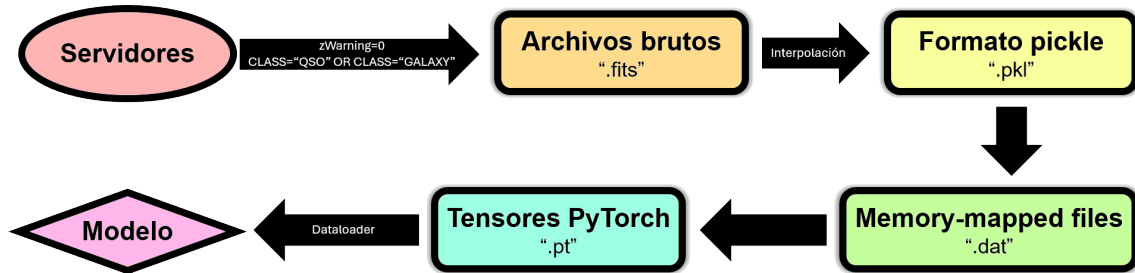


Figure 20: Diagram of data retrieval and preprocessing pipeline.

Given the massive dataset volume, structured preprocessing was essential to optimize storage, loading throughput, and numerical stability.

3.2.1 Downloading Raw Files

Practical execution began with building iterative **SQL** queries sent to the **CAS** server. To avoid server overload on millions of entries, queries were batched by requesting objects sorted by redshift z , using the maximum z of each batch as the lower bound for the subsequent query.

The resulting object keys were used to download raw `.fits` files from the `SAS` server. Implementing parallelized downloads reduced execution time from weeks to two days.

Models were trained exclusively on Galaxy and QSO (*Quasi-Stellar Object*) spectra filtered by `zWarning` = 0 to guarantee quality.

3.2.2 Data Manipulation

Dimensional Handling and Uniformity

Next, raw `.fits` files were batch-processed to extract `flux`, wavelength (`loglam`), and target redshift `z`. To homogenize sample lengths, linear interpolation inserted intermediate points:

$$\lambda_{\text{new}} = \frac{\lambda_i + \lambda_{i+1}}{2}, \quad f_{\text{new}} = \frac{f_i + f_{i+1}}{2},$$

resampling each spectrum to 5000 uniform flux and wavelength points. Interpolation density was emphasized around high-flux-gradient regions to preserve spectral line profiles.

Batches were stored in Python dictionary pickle files (`.pkl`) containing `wavelength`, `flux`, and `redshift` keys, after which raw `.fits` files were cleaned up.

To scale model evaluation across millions of spectra (3+ million spectra at 10,001 data points each), loading efficiency became a primary bottleneck.

We converted data to memory-mapped binary files (`.dat`). Memory-mapped files allow direct disk-to-memory access without loading entire datasets into RAM simultaneously, enabling fast random access for validation sets.

Once memory-mapped files were constructed, data were tensorized into finalized PyTorch formats.

Construction of the Training Dataset

Stellar spectra were excluded (removing 300,000 low-redshift sources), leaving a raw redshift distribution that remained highly non-uniform.

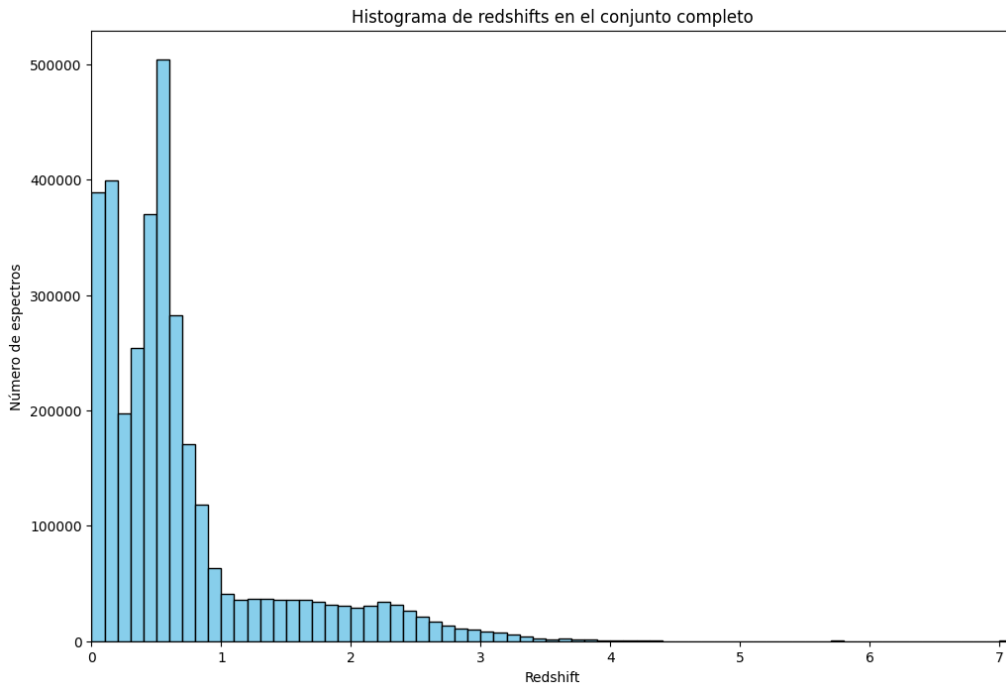


Figure 21: Distribution of sample count as a function of redshift across 3,372,890 spectra.

To prevent model bias toward heavily sampled low-redshift ranges, a balanced sampling strategy was designed. An overall **training** dataset of 504,724 samples (15% of total dataset size) was constructed by stratified sampling across redshift bins: (0, 0.1), (0.1, 1), (1, 2), (2, 3), (3, 4), and (4, 8), taking all available samples for $z \geq 4$.

Using the training set, a **scaler** (normalizer) was fit to standardize mean and variance of flux and wavelength channels before passing tensors into neural network models. Pre-processed data were serialized to PyTorch tensor format (`.pt`).

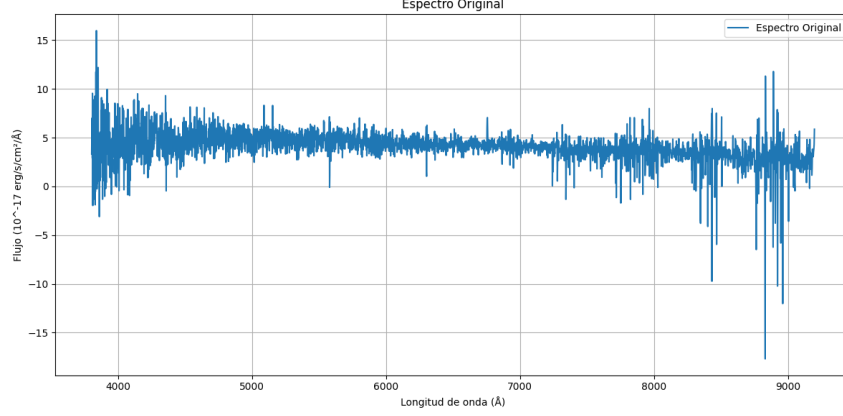


Figure 22: Unnormalized data.

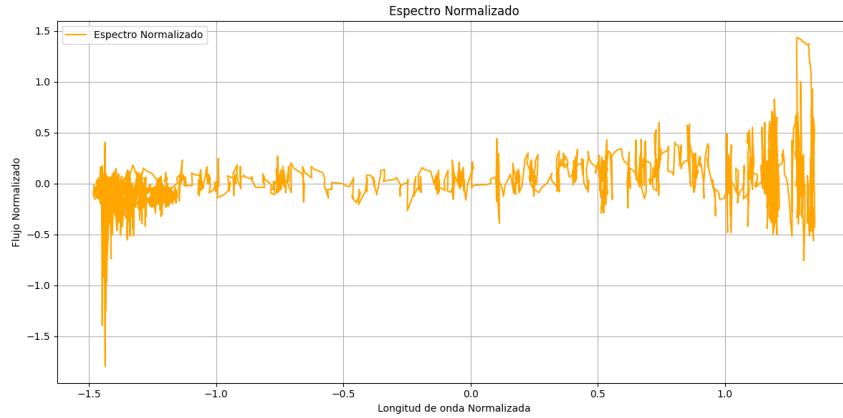


Figure 23: Sample normalized with the scaler.

The training dataset was duplicated into two versions labeled `_600k` and `_1M`. The baseline **validation** dataset (95,276 spectra) was sampled independently without overlap to form the `_600k` suite.

To test regression performance on high redshift ($z \geq 4$), an expanded validation set (595,276 samples) was created including all $z \geq 4$ sources alongside disjoint low-redshift samples, forming the `_1M` suite.

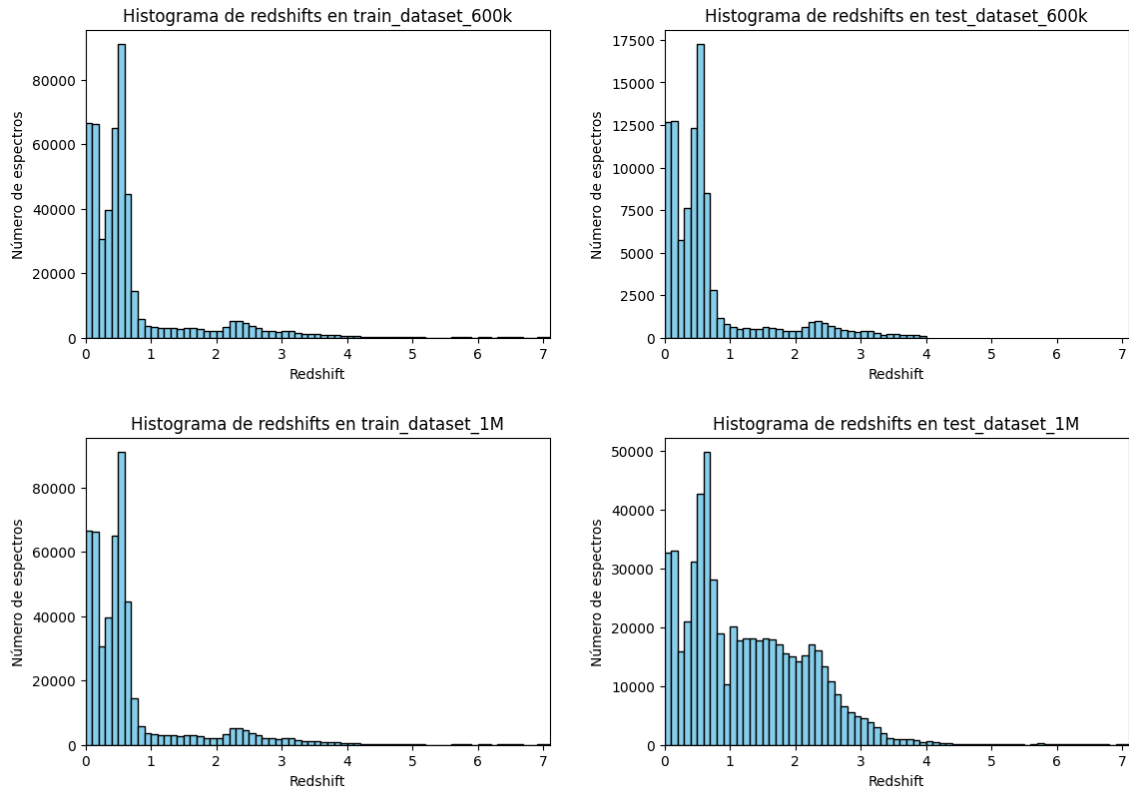


Figure 24: Histograms showing redshift distribution across different datasets.

Training sets were used for forward propagation, loss evaluation, backpropagation, and weight updating (as detailed in Chapter 2). Validation sets monitored overfitting and governed model checkpoint saving based on validation loss reduction.

All datasets were stored as serialized `.pt` files. Finally, auxiliary test datasets (`stars_dataset` and `NASA_dataset`) were preprocessed following an identical pipeline for out-of-distribution evaluation.

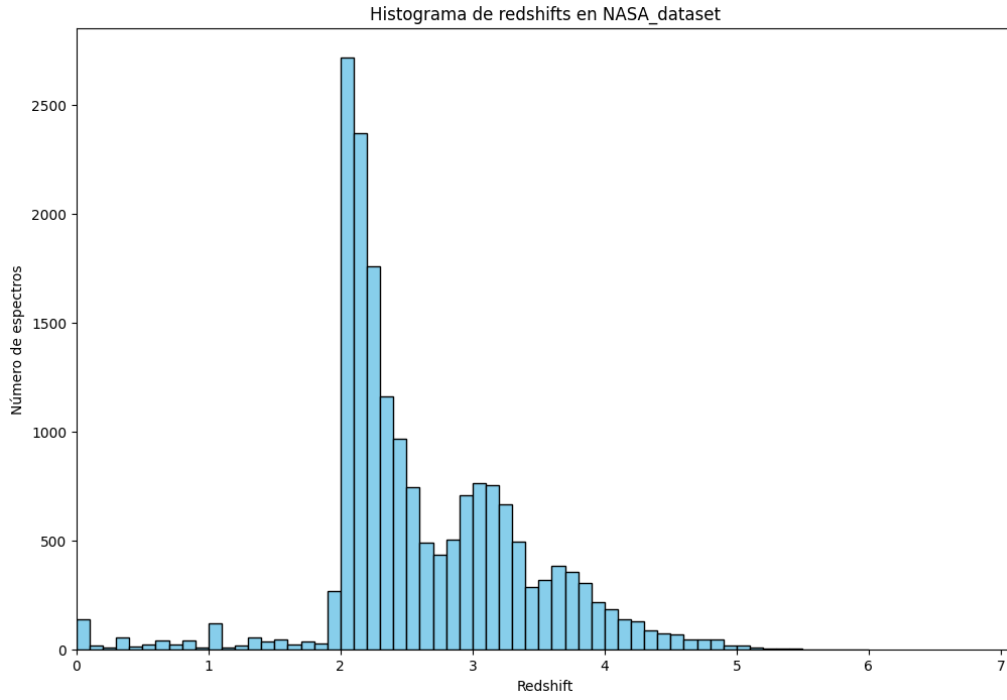


Figure 25: Redshift distribution of the 18,255 samples in the NASA dataset. The distribution for `stars_dataset` is omitted as redshift is bounded above by 0.00462.

For NASA `.fits` files, wavelength and flux extraction followed custom formatting scripts available in the codebase.

Chapter 4

Problem Modelling

4.1 Global Structure

All notebook implementations dedicated to models follow a unified structural workflow, summarized in Figure 26.

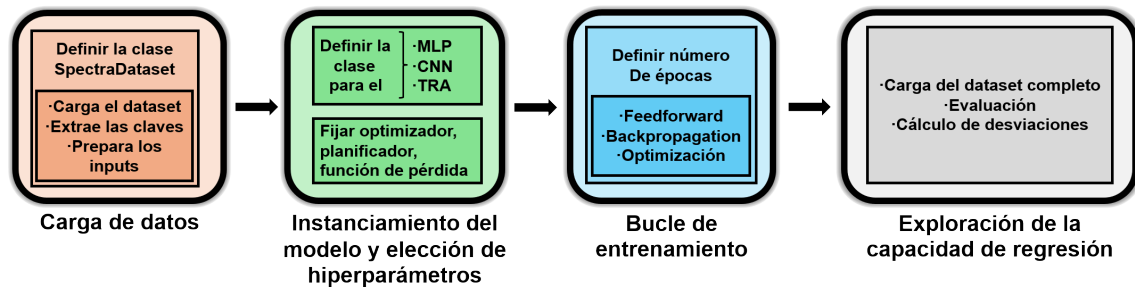


Figure 26: Visual summary of the codebase execution workflow.

This section details each component phase and its technical specifications.

4.1.1 Data Loading

The training pipeline begins with dataset loading. Given high memory overhead, a custom PyTorch dataset wrapper encapsulates pre-saved tensor objects. Dataset length is evaluated by querying tensor shapes directly rather than traversing elements in memory.

During execution, sample vectors (flux and wavelength) are lazily loaded on demand per batch index. This lazy loading mechanism ensures only data required for the active batch reside in GPU memory.

Spectra are grouped into fixed mini-batches of size 32, balancing gradient estimation noise against GPU memory bandwidth.

4.1.2 General Hyperparameters

Model instantiation automatically detects hardware acceleration (CUDA). Utilizing CUDA streams transfers tensors and network parameters directly to GPU memory, enabling parallel kernel operations.

The loss function selected is **Mean Absolute Error (MAE, L1 Loss)**, measuring the average magnitude of absolute discrepancies between predicted and target values without squaring residuals.

Optimization relies on the **Adam** optimizer with an initial learning rate of 10^{-4} and weight decay. Adam dynamically adapts per-parameter learning rates using first and second moment estimates [27].

A learning rate scheduler (**ReduceLRonPlateau**) monitors validation metrics. When validation loss plateaus for a specified patience threshold, the learning rate is scaled down by a predefined factor, reducing step sizes near loss minima.

4.1.3 Training and Validation

Network models support state checkpointing, preserving weights, optimizer states, and scheduler parameters to allow resuming training seamlessly. Training loops iterate over mini-batches using standard forward passes, loss evaluation, backpropagation, and parameter updating.

During validation, forward passes evaluate held-out validation batches without gradient computation. Validation loss determines learning rate adjustments and model checkpointing.

Upon completing training epochs, training and validation error trajectories are plotted, followed by final evaluation on train and test partitions.

4.1.4 Regression Power

For full dataset evaluation, dataset loaders partition data into two separate splits (denoted by `_1` and `_2` path suffixes) containing pre-saved `.pt` tensors (flux, wavelength, redshift).

Predictions and target values are accumulated across full dataset splits without updating model parameters. Evaluation metrics include total dataset MAE, high-redshift error metrics, and counts of negative redshift predictions (noting that target downloads were strictly bounded by $z > 0$).

Evaluation includes two-dimensional density heatmaps comparing ground truth vs. predicted redshift values. Ideal performance aligns predictions along the identity diagonal $y = x$, quantified mathematically by the **Pearson correlation coefficient** [29].

4.2 Model Architectures

4.2.1 Multilayer Perceptron

Each spectrum input consists of 5000 flux values and 5000 wavelength values concatenated into a 10,000-element vector. Linear layers process this 1D signal without assuming explicit spatial correlations, relying on depth to extract non-linear feature interactions.

The architecture comprises seven sequential blocks of linear transformations and ReLU activations. Dimension transitions proceed from 10,000 to 2048, 1024, 512, 256, 128, and 64 neurons, progressively compressing representations into lower-dimensional feature spaces.

Dropout ($p = 0.5$) is applied prior to the output layer to prevent co-adaptation of neurons.

A final linear layer projects the 64-dimensional feature vector into a scalar prediction representing cosmological redshift z .

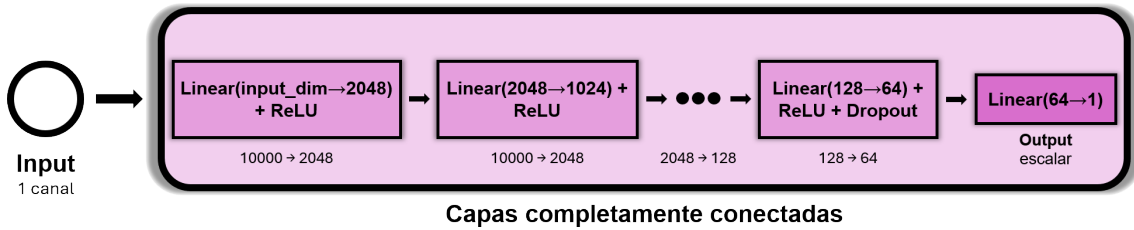


Figure 27: Visual structure of the MLP architecture.

4.2.2 Convolutional Neural Network

The 1D Convolutional Neural Network (CNN) processes two-channel spectral inputs (flux and wavelength) of length 5000.

Five sequential 1D convolutional blocks [13] (kernel size 3, padding 1) are applied with

ReLU activations and max-pooling (kernel size 2). Channel depth expands progressively from 2 to 256 filters, while max-pooling reduces sequence length by a factor of 2 per block.

The resulting feature tensor of shape $(256, \frac{5000}{32})$ (where $5000/32 = 156$) is flattened into a 39,936-element vector, passed through a 128-unit linear layer with ReLU activation, and mapped to the final output scalar.

Five CNN variants were investigated in the project codebase:

- (i) `...0.ipynb`: baseline 1D convolutional network architecture.
- (ii) `...1.ipynb`: adds dropout prior to the final linear prediction layer.
- (iii) `...2.ipynb`: expands convolutional layer depth.
- (iv) `...3.ipynb`: incorporates a Multi-Head Attention mechanism [11] based on [3] between convolutional blocks and linear layers.
- (v) `...4.ipynb`: evaluates inter-block dropout across multiple convolutional stages.

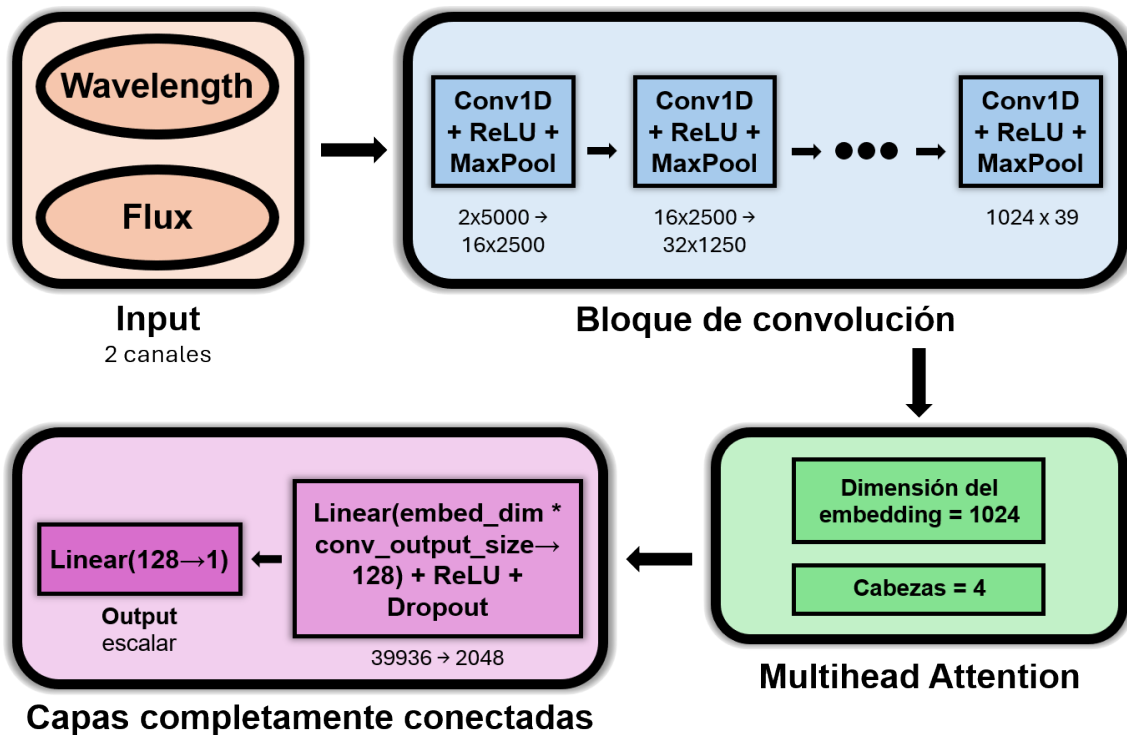


Figure 28: Visual diagram of the CNN architecture (version 3 with Multi-Head Attention).

4.2.3 Transformer

The Transformer model (**TRA**) receives two-channel spectral inputs (2×5000). Initial 1D convolutions project inputs into a feature representation of dimension $d \times L$, where embedding dimension $d = 128$ and sequence length $L = 5000/16 = 312$.

Sinusoidal positional encodings (PE) of shape $1 \times L \times d$ are added to feature representations:

$$\begin{cases} \text{pe}_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d}}\right), & pos = 0, \dots, L-1 \\ \text{pe}_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d}}\right), & i = 0, \dots, \frac{d}{2} - 1 \end{cases}$$

The positional-encoded tensor is passed through a Transformer Encoder [14] consisting of 4 encoder layers with 8 attention heads.

Sequence representations are pooled across sequence positions into a single vector and passed to linear layers to produce the scalar redshift prediction.

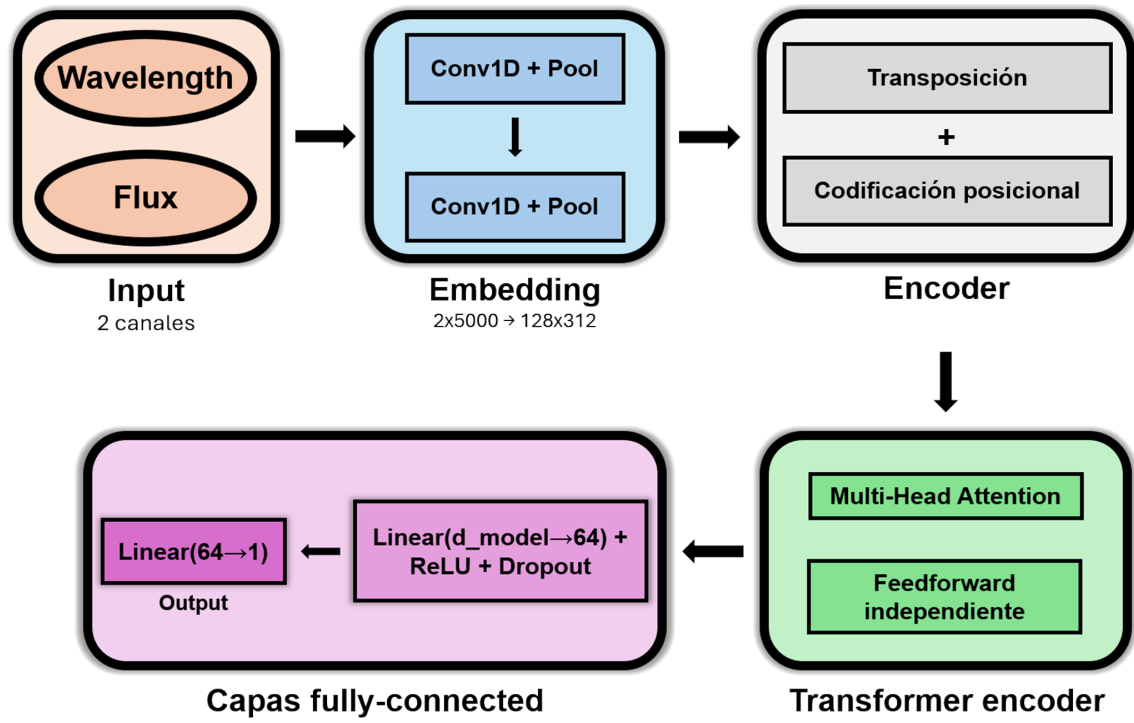


Figure 29: Visual summary of the TRA architecture.

Chapter 5

Results

5.1 Comparative Analysis

This section evaluates trained models on the full dataset of 3,372,890 SDSS spectra (comprising galaxy and quasar samples used in training).

The final cell of each notebook allows evaluating and visualizing random sample spectra from the [spectrums](#) directory.

Results are presented via visual density plots and quantitative error metrics evaluating ground truth vs. predicted values.

5.1.1 Training Datasets

We first evaluate performance differences between models trained on the `_600k` dataset vs. the `_1M` dataset.

Evaluations compare models trained on `_600k` against those trained on `_1M` at epoch 30.

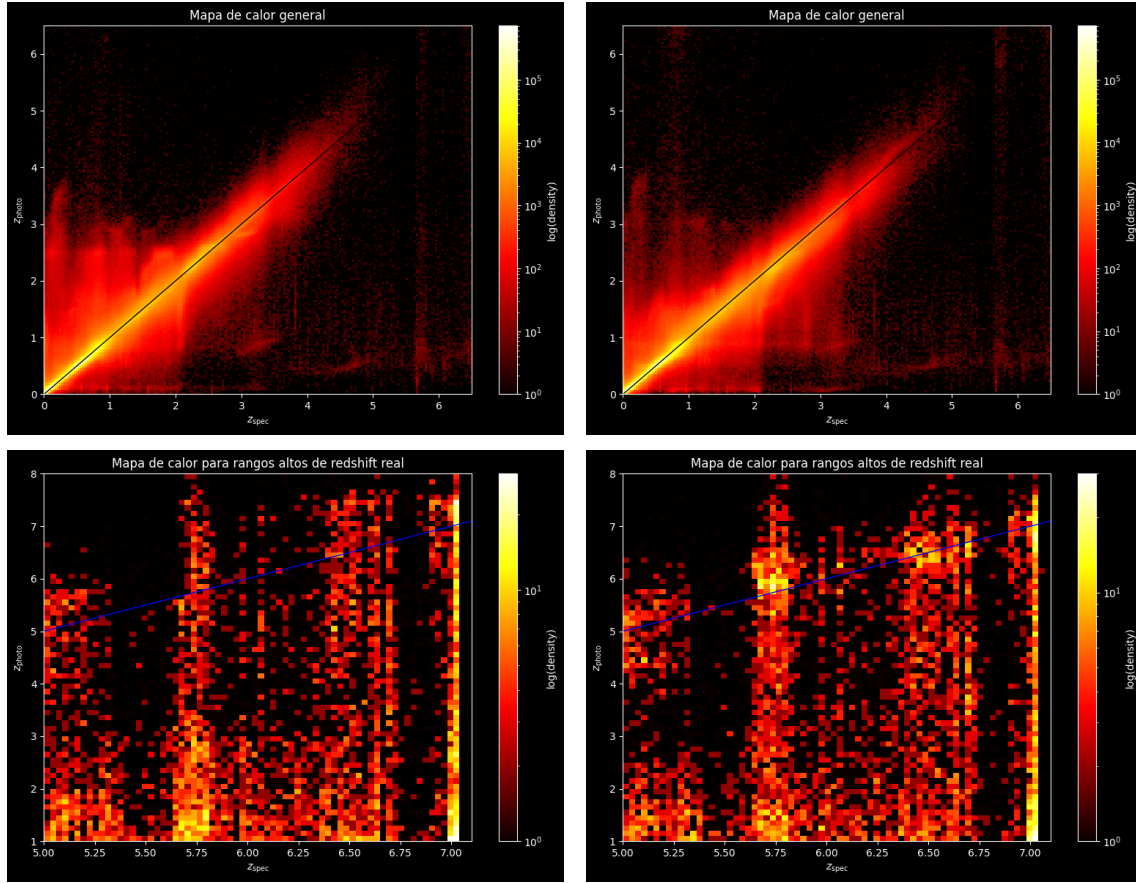


Figure 30: Heatmaps showing performance differences when using _600k (left) vs _1M (right) datasets.

Figure 30 displays point density (z_{spec}, z_{photo}) per pixel. Upper panels cover full redshift ranges; lower panels focus on high-redshift regimes.

While metrics across mid-ranges are comparable, the _1M model shows clear accuracy gains for $z_{spec} > 5$ without inducing overfitting.

This improvement is evident when isolating the **TRA** model architecture:

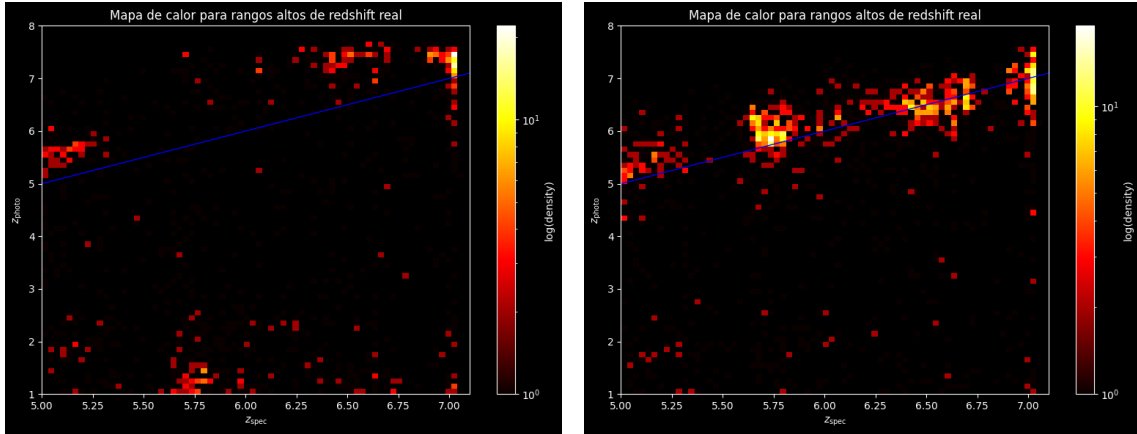


Figure 31: Heatmaps comparing TRA 600k (left) vs TRA 1M (right) at high redshifts.

In Figure 31, diffuse high-redshift predictions align linearly along the identity diagonal. Mean absolute error drops from 3.7 (`_600k`) to 1.8 (`_1M`).

Consequently, subsequent evaluations focus on `_1M`-trained models.

5.1.2 The MLP Model

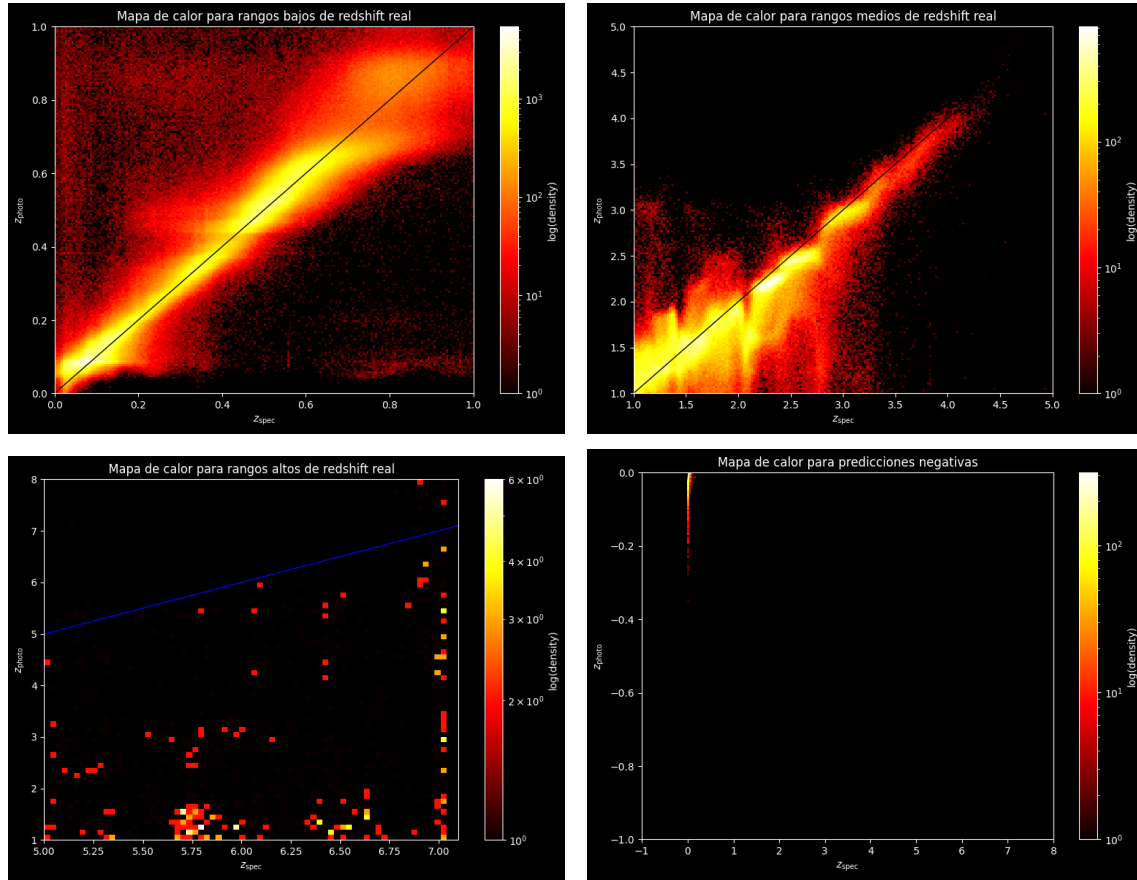
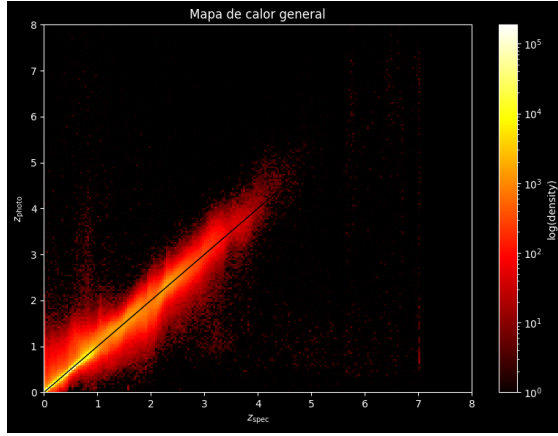


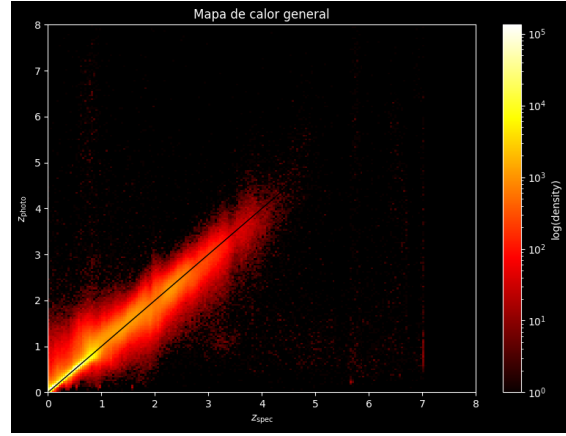
Figure 32: Heatmaps evaluating the MLP model on the full dataset across redshift ranges.

The baseline MLP model yields a Pearson correlation coefficient of 0.477, MAE of 0.128, and 2,181 negative redshift predictions, indicating limited capacity to extract complex non-linear spectral patterns.

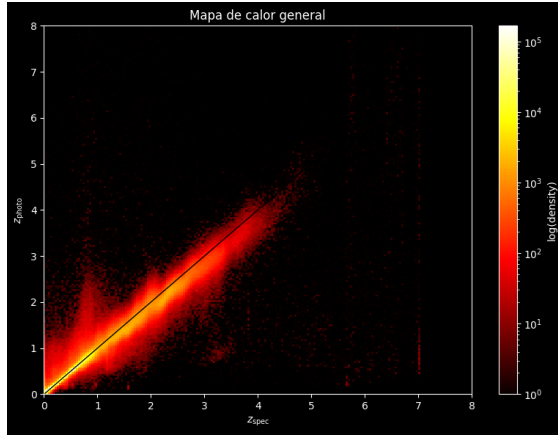
5.1.3 The CNN Models



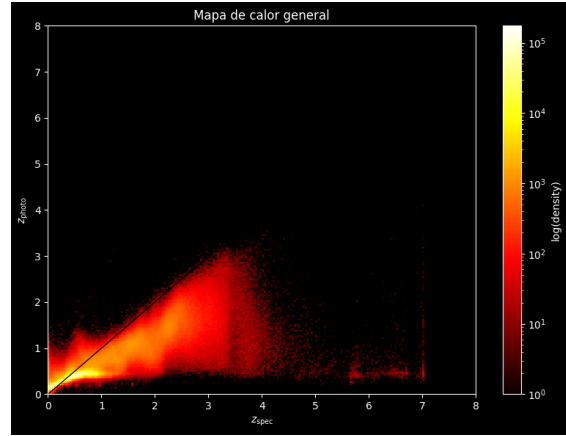
(a) Linear Correlation: 0.011. MAE: 0.098



(b) Linear Correlation: 0.009. MAE: 0.134



(c) Linear Correlation: 0.005. MAE: 0.153



(d) Linear Correlation: 0.001. MAE: 0.356

Figure 33: Results across CNN model variants: $(a) \equiv 0$, $(b) \equiv 1$, $(c) \equiv 2$, and $(d) \equiv 4$.

Baseline CNNs show reduced dispersion over MLPs. However, adding aggressive inter-layer dropout across all convolutional stages degrades performance substantially (Figure 33d).

Conversely, CNN variant 3 incorporates `MultiHeadAttention` [11], achieving a Pearson correlation of 0.967, MAE of 0.073, and 565 negative predictions (Figure 34).

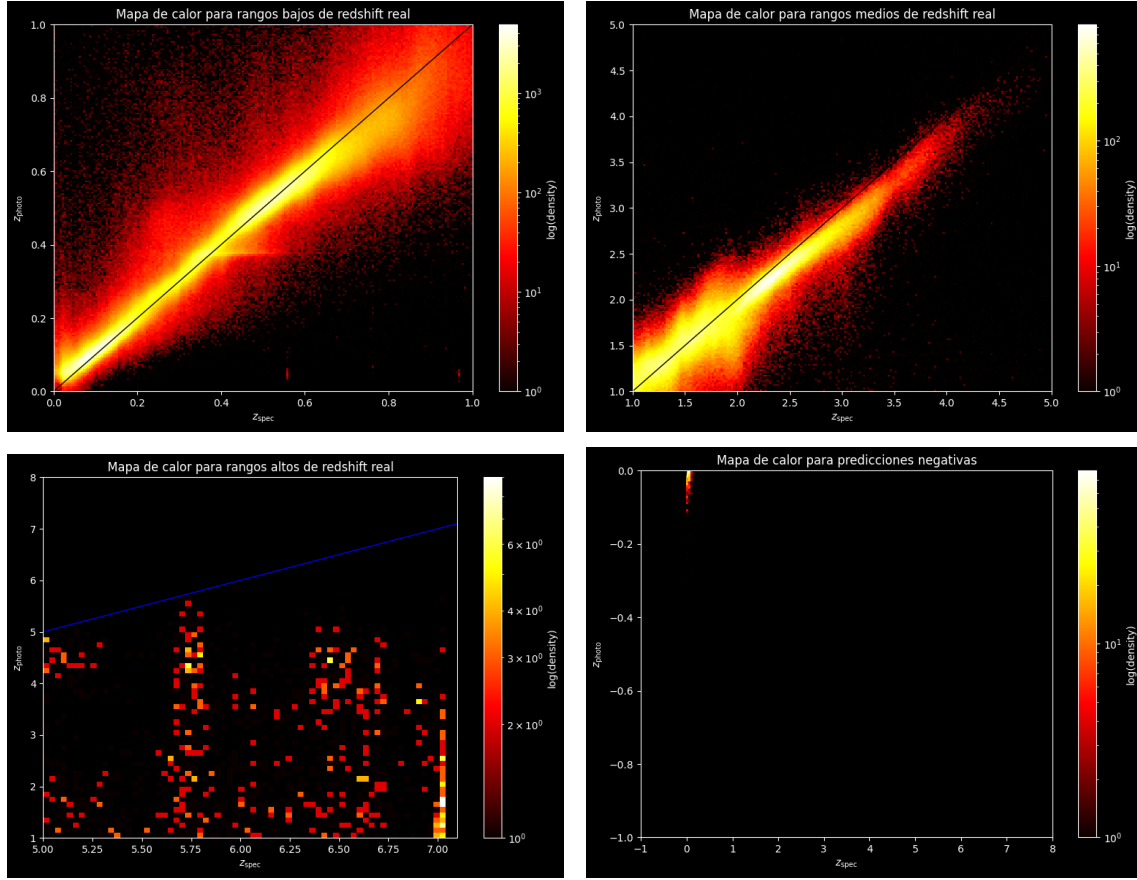


Figure 34: Heatmaps evaluating CNN Variant 3 (with Attention) on the full dataset across redshift ranges.

Limitations remain at $z \geq 5$, where MAE rises to 4.007.

5.1.4 The Transformer Model

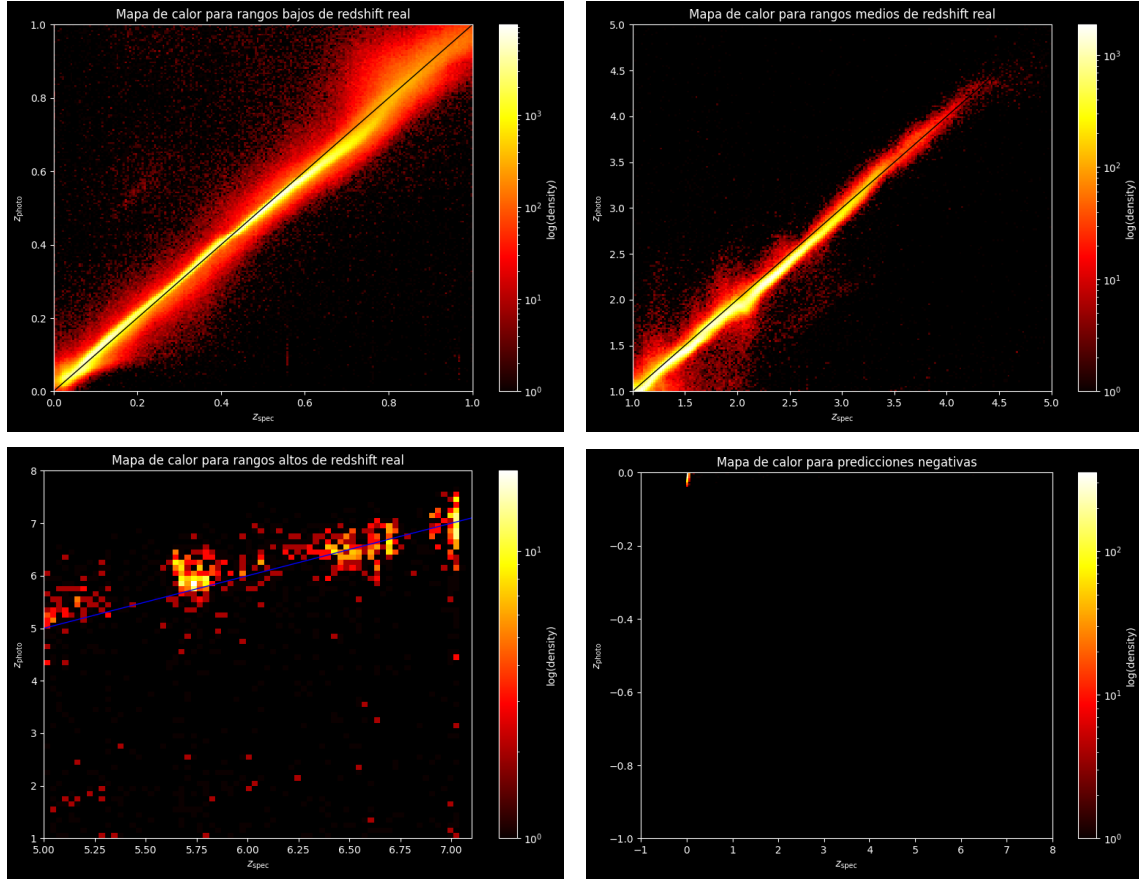


Figure 35: Heatmaps evaluating the TRA Transformer architecture on the full dataset.

The Transformer architecture (TRA) achieves superior generalization, connecting distant emission line peaks across the spectrum. It achieves a Pearson correlation coefficient of 0.965, mean absolute error of 0.039, and 1,570 negative redshift predictions (clustering tightly near $z = 0$).

5.1.5 Influence of Normalization

We evaluated replacing `StandardScaler` [30] with a shape-preserving normalization:

- (i) Zero-centering wavelength vectors.
- (ii) Scaling wavelength vectors to $[-1, 1]$ by maximum absolute value.
- (iii) Normalizing flux to unit peak magnitude.

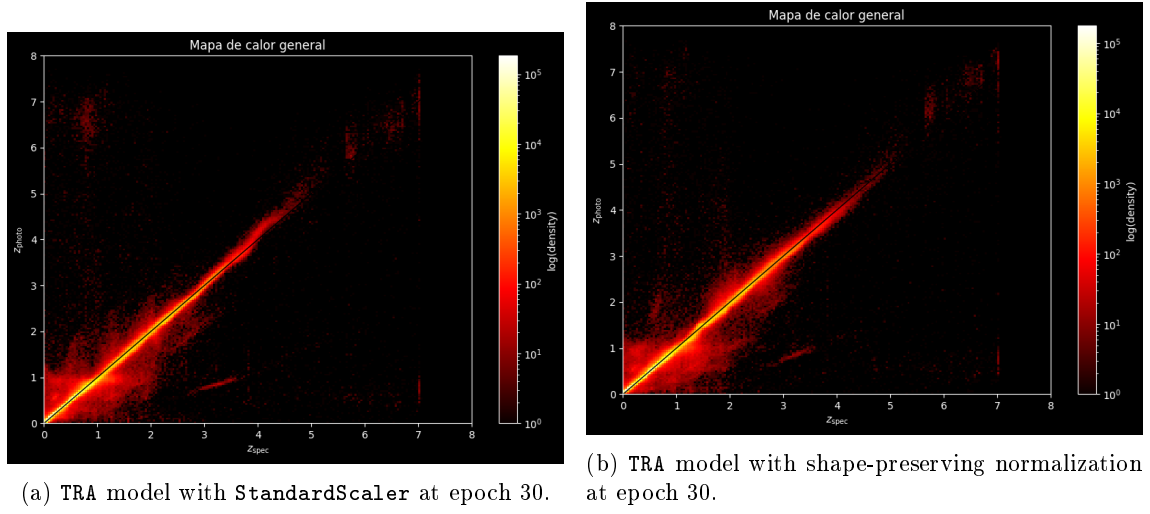


Figure 36: Effect of input normalization choice.

Shape-preserving normalization slightly improves performance (Pearson correlation 0.975, MAE 0.030, 665 negative predictions) while providing robust defense against **data drift**.

5.2 Challenges and Issues

5.2.1 Epochs and Convergence

Extending TRA training beyond 30 epochs on the _1M dataset produced marginal gains (Pearson correlation 0.967, MAE 0.031), indicating a convergence plateau under current hyperparameter settings.

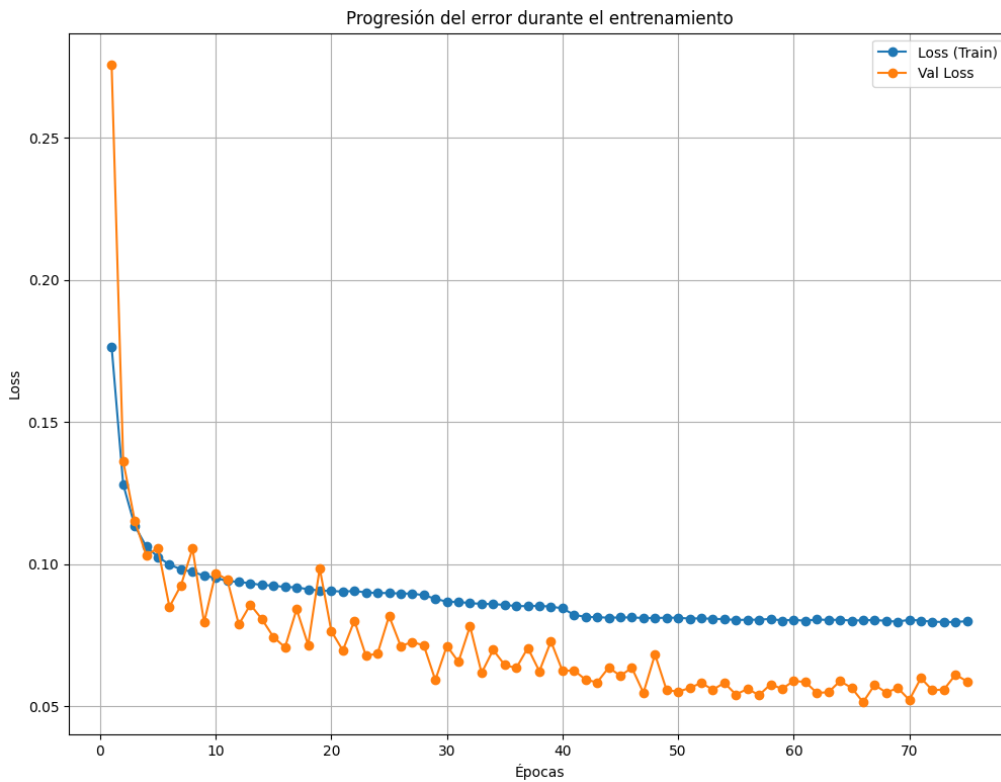


Figure 37: Error evolution of the TRA model across training epochs.

5.2.2 Low Redshift Range

Evaluating TRA on stellar spectra (`stars_dataset`, $z \leq 0.00462$) highlights relative error amplification near $z \approx 0$.

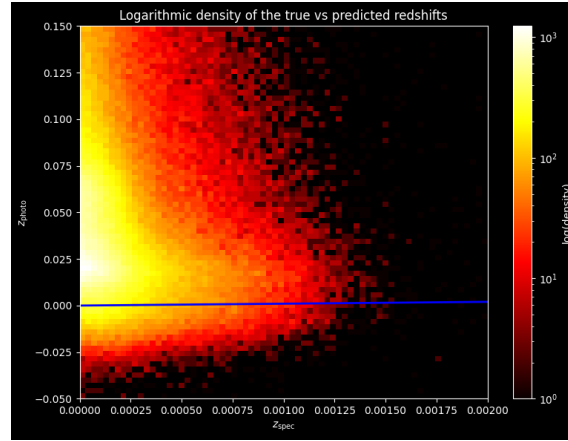


Figure 38: Evaluation of TRA model on stellar dataset.

The model yields Pearson correlation of 0.0884 and MAE of 0.627859. Uniform absolute error metrics produce high relative error at near-zero redshifts, indicating a need for dedicated low-redshift models or adaptive loss weighting.

5.2.3 Data Drift

Out-of-distribution evaluation on NASA/NED spectra exposed data drift sensitivities caused by resolution and instrument variations.

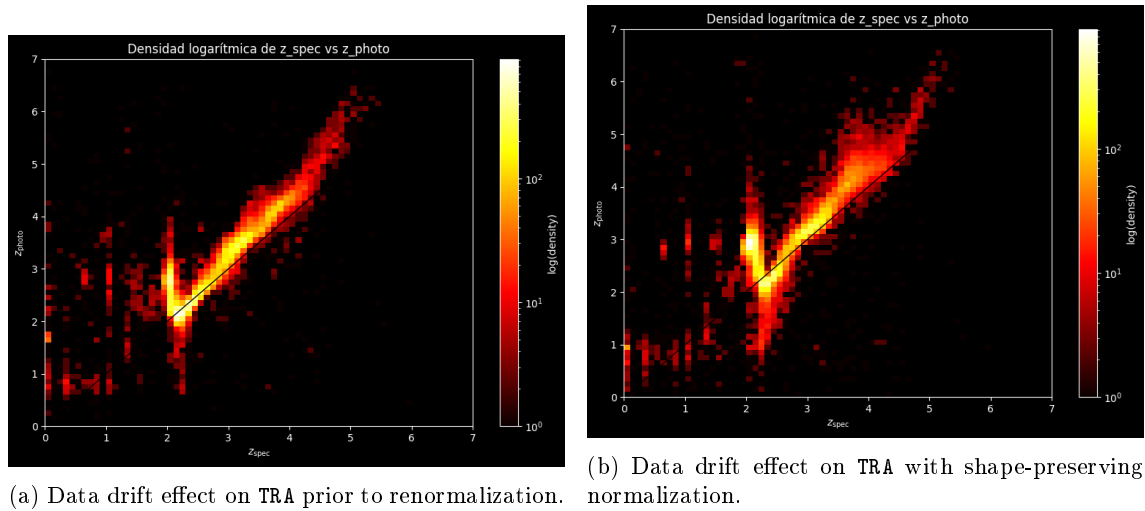


Figure 39: Data drift comparison on NASA/NED dataset.

The unrenormalized model achieves $r = 0.812$ and $\text{MAE} = 0.335$, whereas renormalized input yields $r = 0.72$ and $\text{MAE} = 0.483$.

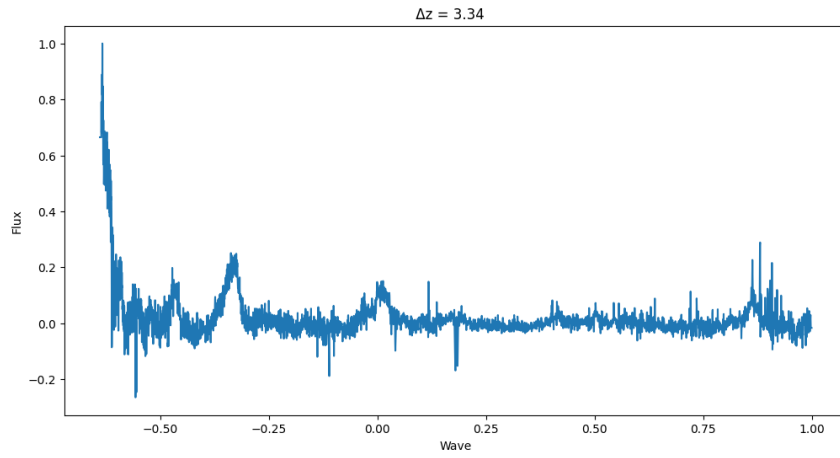


Figure 40: NASA spectrum sample at $z = 2.07$.

Interpolation across lower native resolution spectra (1200×1200 upsampled to 5000) shifted feature density non-uniformly, illustrating domain shift challenges.

5.3 Discussion

Increasing dataset size to $\sim 1\text{M}$ improved high-redshift regression without inducing overfitting or low-redshift degradation.

Multilayer Perceptrons served as baseline validation for the dataset pipeline, though linear feature concatenation proved insufficient for fine spectral line regression.

Convolutional architectures effectively captured local line profiles, while attention mechanisms (`MultiHeadAttention` and full Transformers) resolved global spectral correlations between distant absorption/emission features.

Shape-preserving normalization maintained spectral line ratios across varied input scales, offering improved robustness against domain drift.

Future research directions include adaptive loss functions weighted by redshift magnitude, domain adaptation layers for heterogeneous spectrographs, and dedicated low-redshift sub-networks.

Chapter 6

Conclusions

We conclude this thesis by summarizing the fulfillment of its overall objectives. On the theoretical front, this work explored the physical interpretation of emission lines and redshift cosmology. Furthermore, research into machine learning fundamentals established a solid theoretical foundation.

The accuracy limitations of baseline models (MLP and CNN) prompted exploring alternatives, culminating in adapting Transformer attention mechanisms—originally designed for natural language processing—to astronomical spectra regression.

On the practical front, attention-based models demonstrated utility not only for redshift estimation but also for cross-validating SDSS catalog measurements, providing quality control metrics for cosmological mapping.

Finally, while these architectures represent a first step, machine learning continuously opens new avenues in fields where its applicability was previously unforeseen.

References

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. ISBN: 9780262035613. URL: <https://www.deeplearningbook.org/>.
- [2] J. David Nightingale James Foster. *A Short Course in General Relativity*. 3rd ed. New York: Springer Science, Jan. 2005.
- [3] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. arXiv: 1706.03762 [cs.CL].
- [4] C. O’Raifeartaigh. “The contribution of VM Slipher to the discovery of the expanding universe”. In: *Proceedings of the Origins of the Expanding Universe Conference*. Flagstaff, Arizona, Sept. 2012. URL: <https://philarchive.org/archive/RAITC0-3> (visited on 05/07/2025).
- [5] Matthew D. Schwartz. *Lecture 21: The Doppler Effect*. URL: <https://scholar.harvard.edu/files/schwartz/files/lecture21-doppler.pdf> (visited on 05/26/2025).
- [6] Sloan Digital Sky Survey Collaboration. *SkyServer DR1: Get Image by Plate*. 2003. URL: https://skyserver.sdss.org/dr1/en/tools/getimg/plate.asp?utm_source=chatgpt.com (visited on 05/07/2025).
- [7] Anass El Fares. *A better understanding how a CNN works (Code available)*. Medium. Oct. 20, 2021. URL: https://medium.com/@El_Fares_Anass/a-better-understanding-how-a-cnn-works-code-available-e880f9a338dc (visited on 05/28/2025).
- [8] Swinburne University of Technology. *Gravitational Redshift*. COSMOS, Swinburne University of Technology. URL: <https://astronomy.swin.edu.au/cosmos/g/Gravitational+Redshift> (visited on 05/28/2025).
- [9] NASA Science. *Hubble Gravitational Lenses*. NASA Science. URL: <https://science.nasa.gov/mission/hubble/science/science-behind-the-discoveries/hubble-gravitational-lenses/> (visited on 05/28/2025).
- [10] Robert Hogan, Malcolm Fairbairn, and Navin Seeburn. “GAz: A Genetic Algorithm for Photometric Redshift Estimation”. In: (2014). arXiv preprint. arXiv: 1412.5997 [astro-ph.IM].
- [11] PyTorch Contributors. *torch.nn.MultiheadAttention [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html> (visited on 05/28/2025).
- [12] PyTorch Contributors. *torch.nn.Dropout [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html> (visited on 05/28/2025).
- [13] PyTorch Contributors. *torch.nn.Conv1d [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv1d.html> (visited on 05/28/2025).

- [14] PyTorch Contributors. *torch.nn.TransformerEncoderLayer [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.TransformerEncoderLayer.html> (visited on 05/28/2025).
- [15] PyTorch Team. *torch.nn.ReLU*. PyTorch. 2024. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html> (visited on 05/28/2025).
- [16] Sloan Digital Sky Survey Collaboration. *The Sloan Digital Sky Survey*. Apache Point Observatory and the Astrophysical Research Consortium. 2025. URL: <https://classic.sdss.org/home.php> (visited on 05/28/2025).
- [17] Sloan Digital Sky Survey Collaboration. *Understanding the Imaging Data*. 2025. URL: https://www.sdss.org/dr16/imaging/imaging_basics/ (visited on 05/28/2025).
- [18] Sloan Digital Sky Survey Collaboration. *Filters*. 2025. URL: <https://www.sdss4.org/instruments/camera/#Filters> (visited on 05/28/2025).
- [19] Sloan Digital Sky Survey Collaboration. *The Spectroscopic Pipeline*. 2012. URL: <https://www.sdss3.org/dr9/spectro/pipeline.php> (visited on 05/28/2025).
- [20] Sloan Digital Sky Survey Collaboration. *CasJobs – Batch SQL Queries and MyDB*. 2025. URL: <https://casjobs.sdss.org/> (visited on 05/28/2025).
- [21] Sloan Digital Sky Survey Collaboration. *Science Archive Server (SAS) Data Access*. 2025. URL: https://www.sdss.org/dr17/data_access/tools/ (visited on 05/28/2025).
- [22] NASA/GSFC FITS Support Office. *FITS Standard Documentation*. 2025. URL: <https://fits.gsfc.nasa.gov/> (visited on 05/28/2025).
- [23] Sloan Digital Sky Survey Collaboration. *Redshifts, Classifications and Velocity Dispersions*. 2025. URL: <https://www.sdss.org/dr17/algorithms/redshifts/> (visited on 05/28/2025).
- [24] Sloan Digital Sky Survey Collaboration. *BOSS Pipeline Documentation*. 2022. URL: <https://www.sdss.org/dr18/software/pipelines/boss/> (visited on 05/28/2025).
- [25] NASA/IPAC Extragalactic Database. *NASA/IPAC Extragalactic Database: Home*. 2025. URL: <https://ned.ipac.caltech.edu/> (visited on 05/28/2025).
- [26] Astroquery Developers. *NED Queries (astroquery.ipac.ned)*. n.d. URL: <https://astroquery.readthedocs.io/en/latest/ipac/ned/ned.html> (visited on 05/28/2025).
- [27] PyTorch Contributors. *Adam — PyTorch 2.7 documentation*. n.d. URL: <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html> (visited on 05/28/2025).
- [28] PyTorch Contributors. *torch.optim — PyTorch 2.7 documentation*. n.d. URL: <https://pytorch.org/docs/stable/optim.html> (visited on 05/28/2025).
- [29] NumPy Developers. *numpy.corrccoef — NumPy v2.2 Manual*. n.d. URL: <https://numpy.org/doc/2.2/reference/generated/numpy.corrccoef.html> (visited on 05/28/2025).
- [30] scikitlearn developers. *sklearn.preprocessing.StandardScaler — scikitlearn documentation*. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html> (visited on 05/28/2025).
- [31] Sloan Digital Sky Survey Collaboration. *SQL in SkyServer*. SDSS SkyServer DR16 Documentation. 2024. URL: https://skyserver.sdss.org/dr16/en/help/docs/sql_help.aspx (visited on 05/29/2025).

- [32] Deepsandhya Shukla. *XOR Problem with Neural Networks: An Explanation for Beginners*. May 2, 2025. URL: <https://www.analyticsvidhya.com/blog/2024/01/xor-problem-with-neural-networks-an-explanation-for-beginners/#h-what-is-the-xor-problem> (visited on 05/30/2025).

Glossary

The following list presents key terms used throughout this thesis alongside the page where they first appear.

Theoretical Framework

Machine Learning (ML), [3](#)

Model, [4](#)

Dataset, [4](#)

Loss function, [4](#)

Steps and epochs, [4](#)

Validation, [4](#)

Artificial Neural Network (ANN), [4](#)

Parameters and hyperparameters, [5](#)

Perceptron, [5](#)

Learning rate, [6](#)

Multilayer Perceptron (MLP), [6](#)

Activation function, [7](#)

ReLU, [7](#)

Backpropagation, [8](#)

Optimizer, [8](#)

Scheduler, [9](#)

Convolutional Neural Network (CNN), [10](#)

Receptive field, [10](#)

Kernel, [11](#)

Stride, [12](#)

Padding, [12](#)

Pooling, [13](#)

Transformers (TRA), [14](#)

Embedding, [14](#)

Attention, [14](#)

Attention Head, [15](#)

Overfitting, [16](#)

Underfitting, [17](#)

Dropout, [17](#)

Weight Decay, [17](#)

Redshift, [18](#)

Hubble's Law, [19](#)

Spectra, [21](#)

The Dataset

Sloan Digital Sky Survey (SDSS), [23](#)

Pipeline, [24](#)

SQL, [26](#)

FITS, [26](#)

NED, [27](#)

Quasar (QSO), [29](#)

Scaler / Normalizer, [32](#)

Problem Modelling

Mean Absolute Error (MAE), [36](#)

Linear Correlation Coefficient, [38](#)

Results

Data Drift, [51](#)